

CIS 5357

Computing for Data Analytics

Week 10

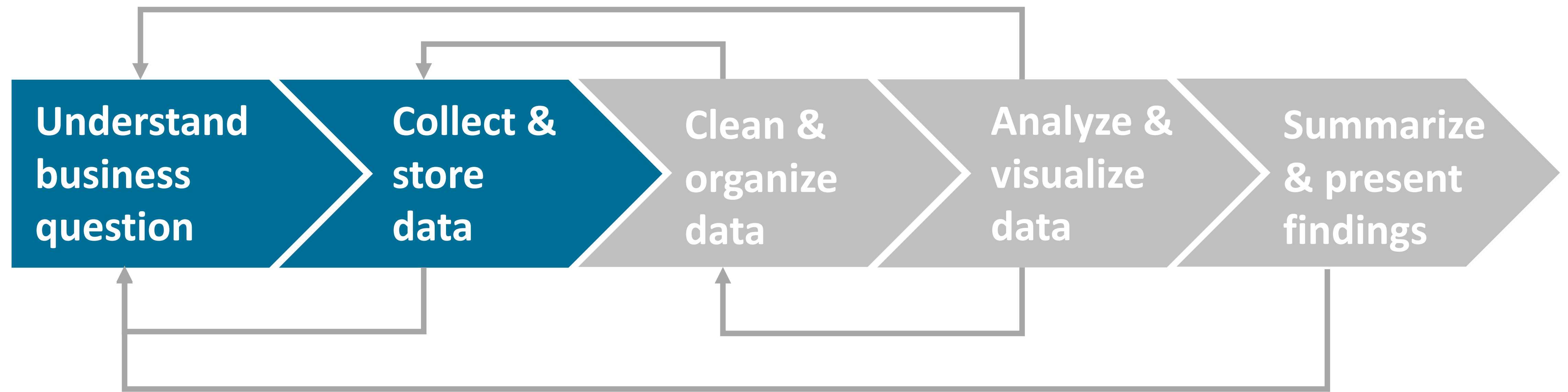
21 MARCH 2024

2 weeks ago

- Data analysis process
- Dig case study
- Jupyter notebooks
- Pandas library: series and DataFrames
- Reading and writing files with pandas

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



Quick review

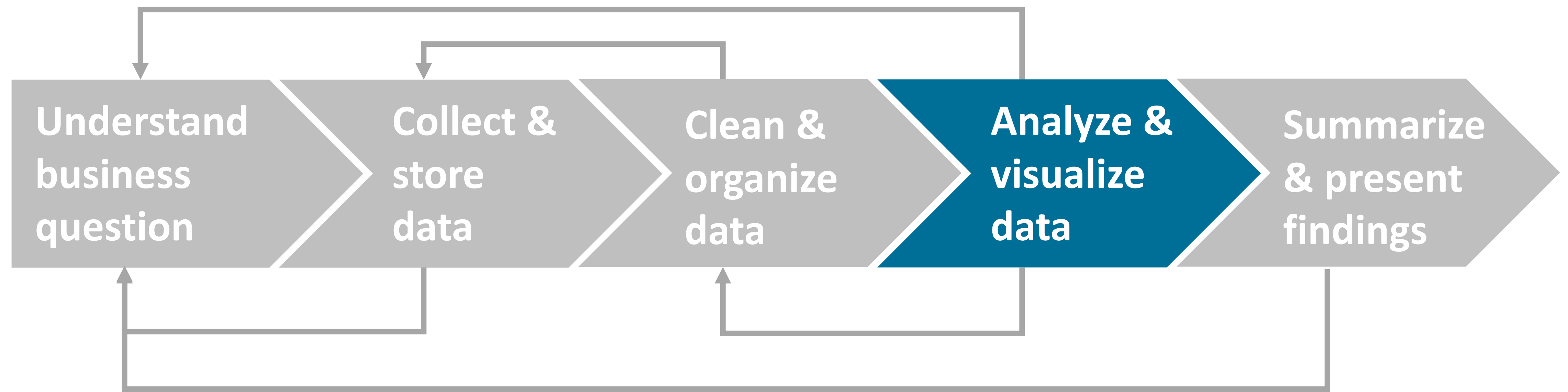
- Import data files to a pandas DataFrame `.read_excel()`, `.read_csv()`, `.read_pickle()`
- Get field names & data types in a pandas DataFrame `.info()`
- Get the number of rows & columns in a DataFrame `.shape`
- See first X records in a pandas DataFrame `.head(X)`
- See last X records in a pandas DataFrame `.tail(X)`
- Write out a pandas DataFrame to a file `.to_excel()`, `.to_csv()`, `.to_pickle()`

This week

- Exploring data with Python's pandas library
- Visualizing data in Python

Data Analytics

the process of analyzing raw data in order to draw out meaningful, actionable insights, which are then used to inform and drive business decisions



This week

- **Exploring data with Python's pandas library**
- Visualizing data in Python

Recall the Dig case study

A New York **restaurant chain** that wants to use **data to inform decisions** at every stage in the value chain as they **scale the business**.

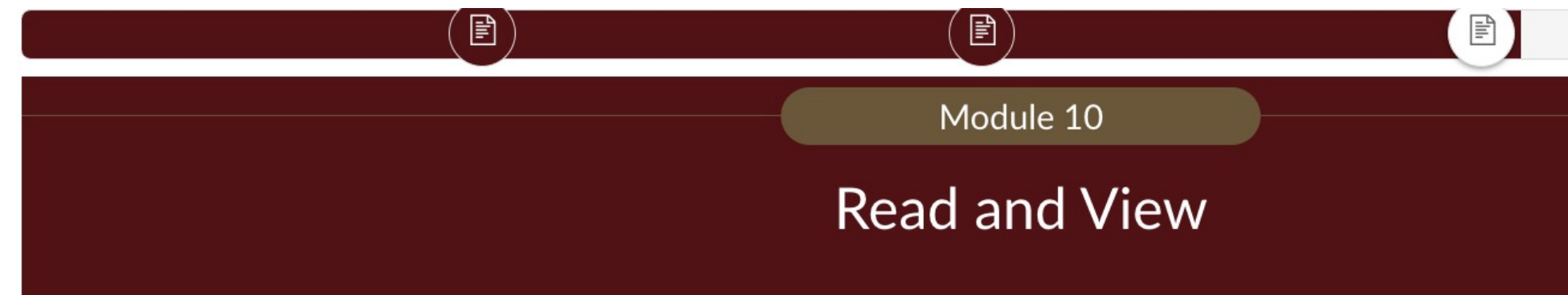
Available data:

- Restaurants.csv—name, address, open data for 9 restaurants
- Items.csv—item name & type for 32 items
- Simplified orders.csv—one line per order in the one-year period spanned by the dataset
- Summarized orders.csv—aggregates the orders file to a daily level—it contains one row per restaurant per day

Let's chat

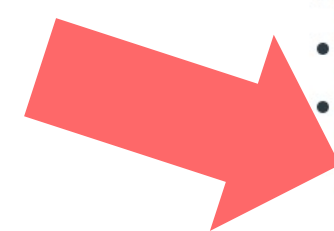
What types of exploratory analysis could we do with the Dig case files?

Download code to follow along



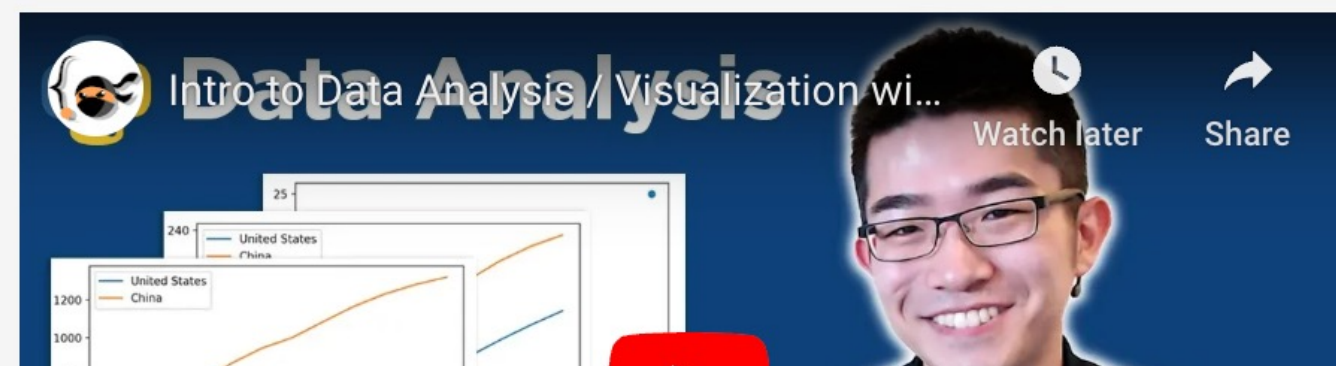
Read

- Textbook chapter 6 pages 193-216, 221-236
- [Exploratory analysis with pandas](#) ↗
- [Pandas: visualization](#) ↗
- [Line charts in Python](#) ↗
- [Practice code](#) ↓ for lecture



View

[Intro to data analysis / visualization in Python](#) ↗



Let's consider summarized orders

This dataset aggregates the orders file to a daily level—it contains one row per restaurant per day

```
df_summarized_orders.head()
```

✓ 0.0s

	RESTAURANT_NAME	DATE	NUM_ORDERS	PERC_DELIVERY
0	Bryant Park	2018-01-01	373	0.0
1	Bryant Park	2018-01-02	789	0.0
2	Bryant Park	2018-01-03	818	0.0
3	Bryant Park	2018-01-04	782	0.0
4	Bryant Park	2018-01-05	719	0.0

Descriptive statistics

For numeric fields pandas has a **describe()** function available, which calculates summary statistics

```
df_summarized_orders.describe()
```

✓ 0.0s

	NUM_ORDERS	PERC_DELIVERY
count	2806.000000	2806.000000
mean	850.756949	0.116700
std	195.490367	0.055078
min	200.000000	0.000000
25%	739.000000	0.090095
50%	833.000000	0.104963
75%	949.000000	0.152853
max	1396.000000	0.360215

Other functions to summarize data

dataframe.COLUMN_NAME.function()

- `sum()`
- `min()`
- `max()`
- `mean()`
- `median()`
- `std()`
- `corr()`

```
df_summarized_orders.NUM_ORDERS.mean()
```

✓ 0.0s

850.7569493941554

```
df_summarized_orders.NUM_ORDERS.corr(df_summarized_orders.PERC_DELIVERY)
```

✓ 0.0s

-0.19667409216417953

Let's consider detailed orders now

```
df_orders.head()
```

✓ 0.0s

	ORDER_ID	DATETIME	RESTAURANT_ID	TYPE	DRINKS	COOKIES	MAIN	BASE	SIDE_1	SIDE_2
0	O1820060	2018-10-11 17:25:50	R10002	IN_STORE	1.0	2.0	NaN	NaN	NaN	NaN
1	O1011112	2018-05-31 11:35:00	R10003	IN_STORE	0.0	0.0	NaN	NaN	NaN	NaN
2	O752854	2018-04-21 18:12:57	R10001	DELIVERY	0.0	2.0	I0	I7	I15	I14
3	O2076864	2018-11-17 12:50:52	R10005	PICKUP	1.0	0.0	I0	I5	I9	I12
4	O1988898	2018-11-04 18:37:24	R10008	IN_STORE	0.0	0.0	I1	I7	I9	I9

Unique lists

To find all the unique values in a column use **unique()**

```
df_orders.COOKIES.unique()
```

✓ 0.0s

```
array([2., 0., 1.])
```

At most 2 cookies are
purchased in an order

Note that, the **unique()** function returns something called an array, which behaves similarly to a list in Python. However, **to_list()** converts the output to a list, which we're more familiar with.

```
df_orders.COOKIES.unique().tolist()
```

✓ 0.0s

```
[2.0, 0.0, 1.0]
```

```
sorted(df_orders.COOKIES.unique().tolist())
```

✓ 0.0s

```
[0.0, 1.0, 2.0]
```

Use `isin()` function to flag records

A common operation to check whether each entry is one of several values

Create a new column named `ONE_TWO_COOKIES`

Look for records with values 1 or 2 in the `COOKIES` column

```
df_orders['ONE_TWO_COOKIES'] = df_orders.COOKIES.isin([1,2])
```

✓ 0.1s

Python

```
df_orders.head()
```

✓ 0.0s

Python

	ORDER_ID	DATETIME	RESTAURANT_ID	TYPE	DRINKS	COOKIES	MAIN	BASE	SIDE_1	SIDE_2	ONE_TWO_COOKIES
0	O1820060	2018-10-11 17:25:50	R10002	IN_STORE	1.0	2.0	NaN	NaN	NaN	NaN	True
1	O1011112	2018-05-31 11:35:00	R10003	IN_STORE	0.0	0.0	NaN	NaN	NaN	NaN	False
2	O752854	2018-04-21 18:12:57	R10001	DELIVERY	0.0	2.0	I0	I7	I15	I14	True
3	O2076864	2018-11-17 12:50:52	R10005	PICKUP	1.0	0.0	I0	I5	I9	I12	False
4	O1988898	2018-11-04 18:37:24	R10008	IN_STORE	0.0	0.0	I1	I7	I9	I9	False

New column added to the DataFrame

Could also use NumPy package

NumPy (Numerical Python) is an open source Python library that's for working with numerical data in Python

```
import numpy as np
```

✓ 0.0s

Create a Boolean field indicating whether an order included a cookie

```
df_orders['COOKIE_PURCHASE'] = np.where(df_orders['COOKIES'] > 0, True, False)
```

✓ 0.0s

Name of the new column

Logic used to define the column

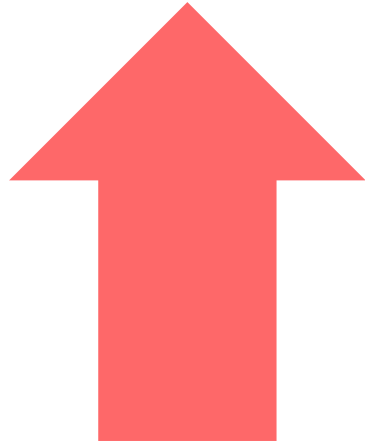
DataFrame now has another column

```
df_orders.head()
```

✓ 0.0s

Python

	ORDER_ID	DATETIME	RESTAURANT_ID	TYPE	DRINKS	COOKIES	MAIN	BASE	SIDE_1	SIDE_2	ONE_TWO_COOKIES	COOKIE_PURCHASE
0	O1820060	2018-10-11 17:25:50	R10002	IN_STORE	1.0	2.0	NaN	NaN	NaN	NaN	True	True
1	O1011112	2018-05-31 11:35:00	R10003	IN_STORE	0.0	0.0	NaN	NaN	NaN	NaN	False	False
2	O752854	2018-04-21 18:12:57	R10001	DELIVERY	0.0	2.0	I0	I7	I15	I14	True	True
3	O2076864	2018-11-17 12:50:52	R10005	PICKUP	1.0	0.0	I0	I5	I9	I12	False	False
4	O1988898	2018-11-04 18:37:24	R10008	IN_STORE	0.0	0.0	I1	I7	I9	I9	False	False



Descriptive statistics non-numeric

```
dataframe.describe(include=["object", "bool"])
```

```
● df_orders.describe(include=["object", "bool"])💡
```

✓ 2.7s

	ORDER_ID	RESTAURANT_ID	TYPE	MAIN	BASE	SIDE_1	SIDE_2	ONE_TWO_COOKIES	COOKIE_PURCHASE
count	2387224	2387224	2387224	2275639	2275639	2275639	2275639	2387224	2387224
unique	2387224	8	3	5	3	8	8	2	2
top	O1820060	R10004	IN_STORE	I0	I6	I10	I10	False	False
freq	1	430860	1713136	693847	918077	497437	497826	1910717	1910717

Let's now consider the TYPE column

```
dataframe.describe(include=["object", "bool"])
```

```
df_orders.describe(include=["object", "bool"])
```

✓ 2.7s

	ORDER_ID	RESTAURANT_ID	TYPE	MAIN	BASE	SIDE_1	SIDE_2	ONE_TWO_COOKIES	COOKIE_PURCHASE
count	2387224	2387224	2387224	2275639	2275639	2275639	2275639	2387224	2387224
unique	2387224	8	3	5	3	8	8	2	2
top	O1820060	R10004	IN_STORE	I0	I6	I10	I10	False	False
freq	1	430860	1713136	693847	918077	497437	497826	1910717	1910717

Counting values

For **categorical** (type object) and **boolean** (type bool) features we can use the **value_counts()**.

It can be applied to any series and will tell you **how often each value occurs in the series**

```
dataframe.COLUMN_NAME.value_counts()
```

```
dataframe['COLUMN_NAME'].value_counts()
```

```
df_orders.TYPE.value_counts()
```

✓ 0.1s

IN_STORE	1713136
PICKUP	401440
DELIVERY	272648

Name: TYPE, dtype: int64

```
df_orders['TYPE'].value_counts()
```

✓ 0.1s

IN_STORE	1713136
PICKUP	401440
DELIVERY	272648

Name: TYPE, dtype: int64

Percentage counts

```
dataframe.COLUMN_NAME.value_counts(normalize=True)
```

```
dataframe['COLUMN_NAME'].value_counts(normalize=True)
```

```
df_orders.TYPE.value_counts(normalize=True)
```

✓ 0.1s

```
IN_STORE    0.717627  
PICKUP      0.168162  
DELIVERY    0.114211  
Name: TYPE, dtype: float64
```

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

We'll resume in
5 minutes...

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

We'll resume in
4 minutes...

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

We'll resume in
3 minutes...

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

We'll resume in
2 minutes...

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

We'll resume in
1 minute...

Challenge 1: 2 drinks

What percentage of orders include exactly 2 drinks?

```
df_orders.DRINKS.value_counts(normalize = True)
```

✓ 0.0s

0.0	9.215005e-01
1.0	6.468727e-02
2.0	1.127418e-02
3.0	1.970490e-03
4.0	3.832904e-04
5.0	5.822663e-05
6.0	1.466138e-05
13.0	1.382359e-05

About 1% of orders
include exactly 2 drinks

```
df_orders.DRINKS.isin([2]).value_counts(normalize=True)
```

✓ 0.0s

False 0.988726

True 0.011274

Name: DRINKS, dtype: float64

```
df_orders['TWO_DRINKS']=(np.where(df_orders['DRINKS'] == 2, True, False))  
df_orders['TWO_DRINKS'].value_counts(normalize=True)
```

✓ 0.0s

False 0.988726

True 0.011274

Name: TWO_DRINKS, dtype: float64

More ways to filter rows

We can also use conditional statements to isolate or filter for certain records

```
#group orders with cookies  
(df_orders.COOKIES > 0).value_counts()
```

✓ 0.0s

```
False    1910717  
True      476507  
Name: COOKIES, dtype: int64
```

```
#group orders with cookies  
(df_orders.COOKIES > 0).value_counts(normalize = True)
```

✓ 0.0s

```
False    0.800393  
True      0.199607  
Name: COOKIES, dtype: float64
```

```
#sum only orders with cookies  
(df_orders.COOKIES > 0).sum()
```

✓ 0.1s

476507

About 20% of orders include at least 1 cookie (n=476,507)

We can even use compound logic

```
#filters orders with a drink AND a cookie  
((df_orders.DRINKS > 0) & (df_orders.COOKIES > 0)).sum()
```

✓ 0.0s

37199

We can even use compound logic

```
#filters orders with a drink AND a cookie  
((df_orders.DRINKS > 0) & (df_orders.COOKIES > 0)).sum()
```

✓ 0.0s

37199

```
#filters orders with a drink OR a cookie  
((df_orders.DRINKS > 0) | (df_orders.COOKIES > 0)).sum()
```

✓ 0.0s

626704

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

We'll resume in
5 minutes...

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

We'll resume in
4 minutes...

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

We'll resume in
3 minutes...

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

We'll resume in
2 minutes...

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

We'll resume in
1 minute...

Challenge 2: drink but no cookie

What percentage of orders include a drink but not a cookie?

```
((df_orders.DRINKS > 0) & (df_orders.COOKIES == 0)).value_counts(normalize = True)
```

✓ 0.0s

False 0.937083

True 0.062917

dtype: float64

Let's take a quick break

We'll resume in

8 minutes...

Let's take a quick break

We'll resume in

7 minutes...

Let's take a quick break

We'll resume in

6 minutes...

Let's take a quick break

We'll resume in

5 minutes...

Let's take a quick break

We'll resume in

4 minutes...

Let's take a quick break

We'll resume in

3 minutes...

Let's take a quick break

We'll resume in

2 minutes...

Let's take a quick break

We'll resume in

1 minute...

This week

- Exploring data with Python's pandas library
- **Visualizing data in Python**

Let's chat

Why do we visualize data?

Popular plotting libraries

Matplotlib

- **fundamental** plotting library that provides a high level of customization
- good for static, non-interactive plots

Seaborn

- provides a high-level interface for drawing informative **statistical** graphics

Plotly

- can create similar charts as Matplotlib & Seaborn, but with the added capability of **interactivity**

Bokeh

- suitable for creating interactive, **web-ready plots**, which can be embedded into web applications

Plotting data library: matplotlib

Begin by loading Python's plotting library

```
import matplotlib.pyplot as plt
```

1

Plotting methods

The **plot()** method on Series and DataFrames allows us to visualize data.

Plotting methods allow for a handful of plot styles (the default is a line plot).

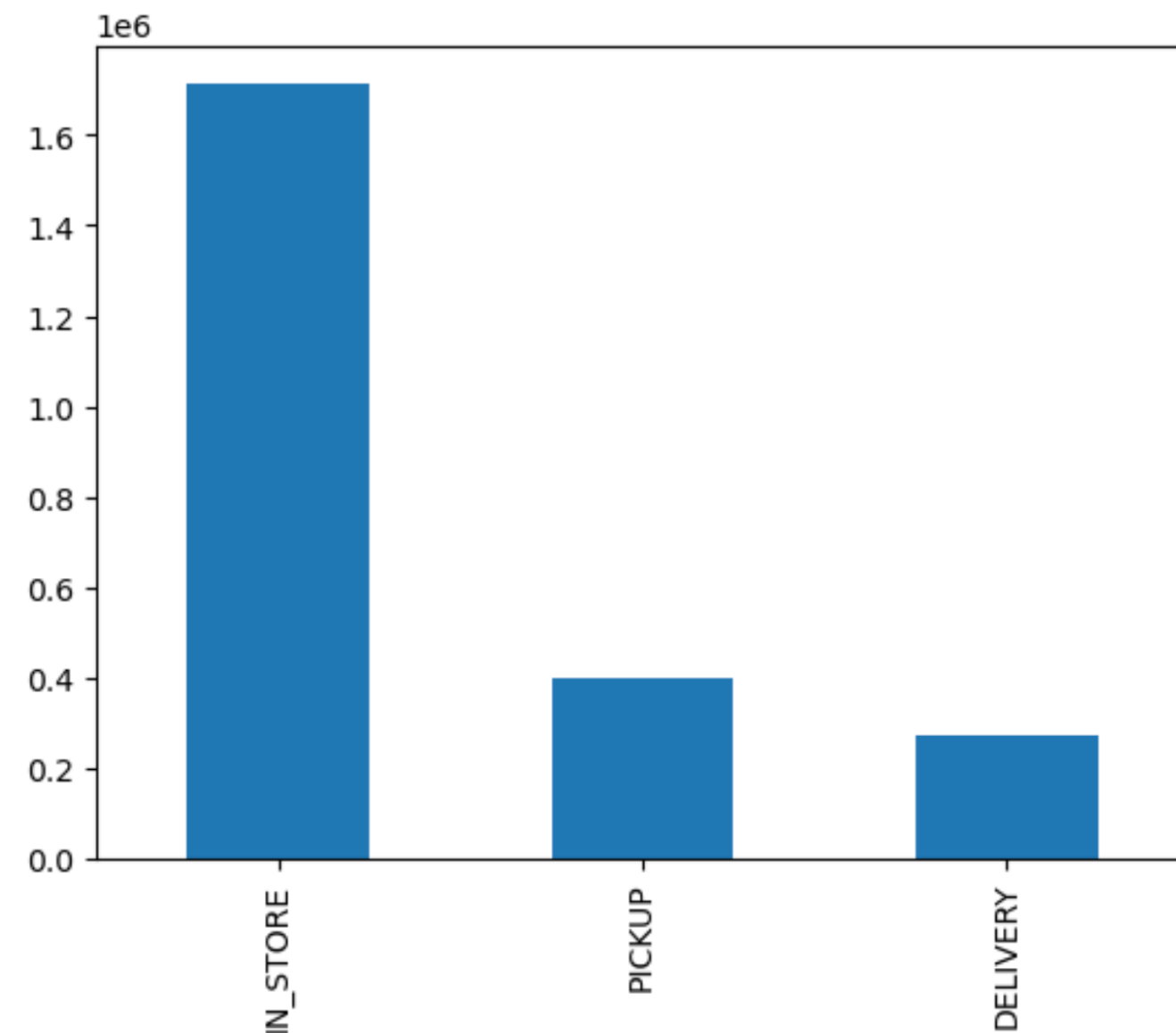
These methods can be provided as the **kind** keyword argument to plot():

- 'bar' or 'barh' for bar plots
- 'hist' for histogram
- 'box' for boxplot
- 'kde' or 'density' for density plots
- 'area' for area plots
- 'scatter' for scatter plots
- 'pie' for pie plots

Bar charts

Useful for categorical information

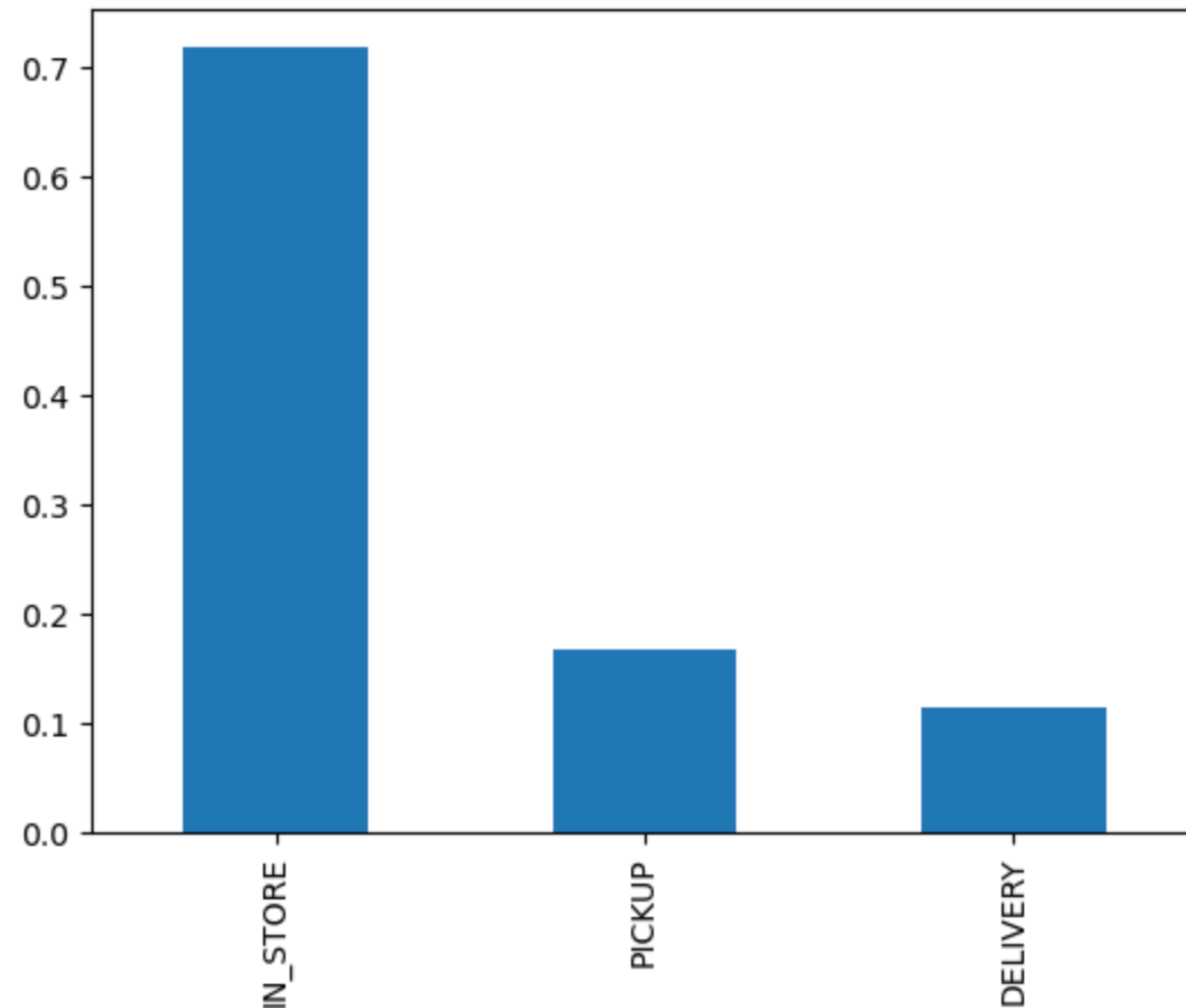
```
df_orders.TYPE.value_counts().plot(kind='bar')
```



- We first access the `TYPE` column as a series
- We then apply `value_counts()` to it, which returns a series with a row index corresponding to the unique entries in that column
- Finally, we plot this series—the row index appears on the horizontal x-axis, and the frequency on the vertical y-axis.

Normalize to plot percentages

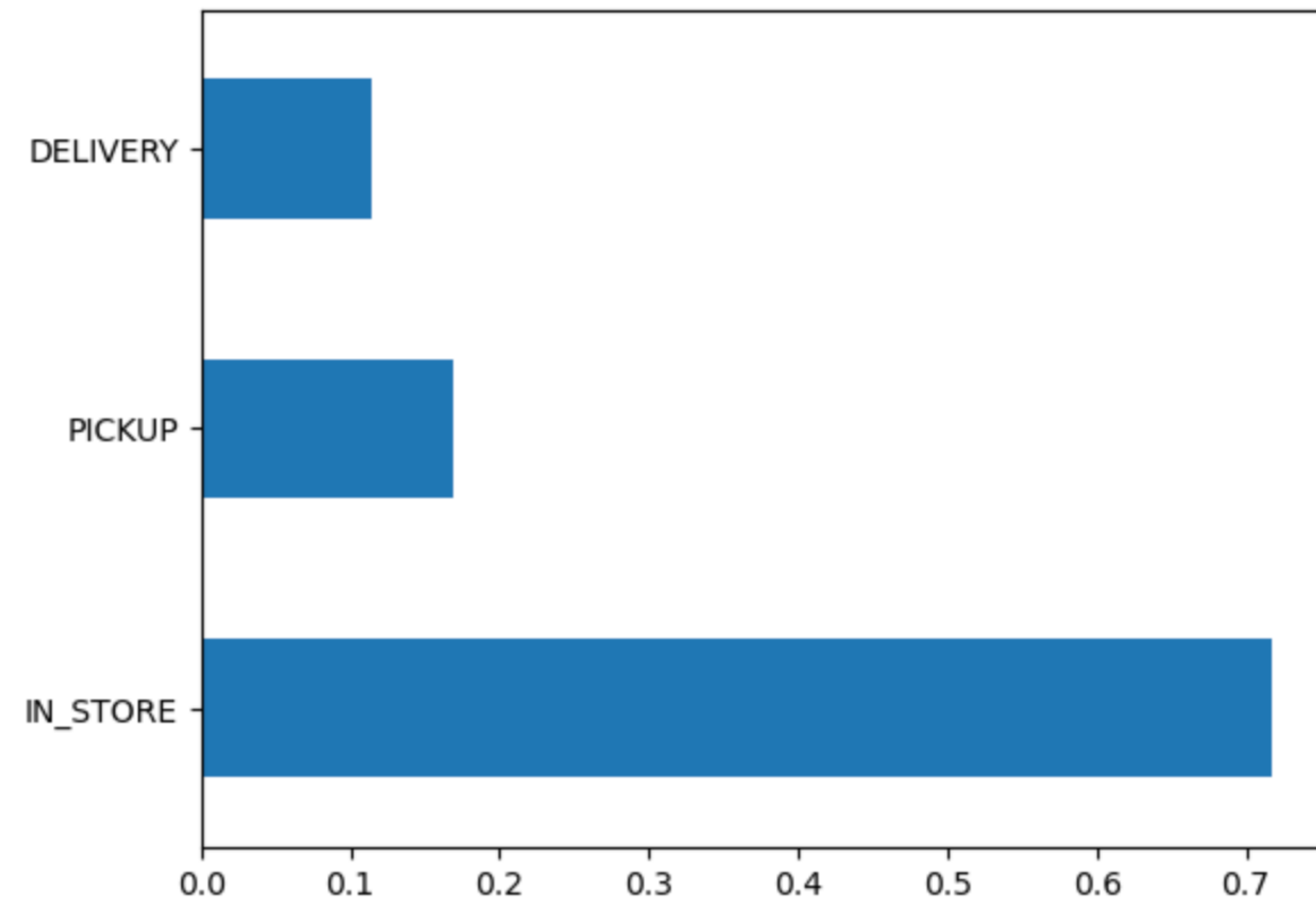
```
df_orders.TYPE.value_counts(normalize=True).plot(kind='bar')
```



Horizontal bar charts

Useful for long category labels

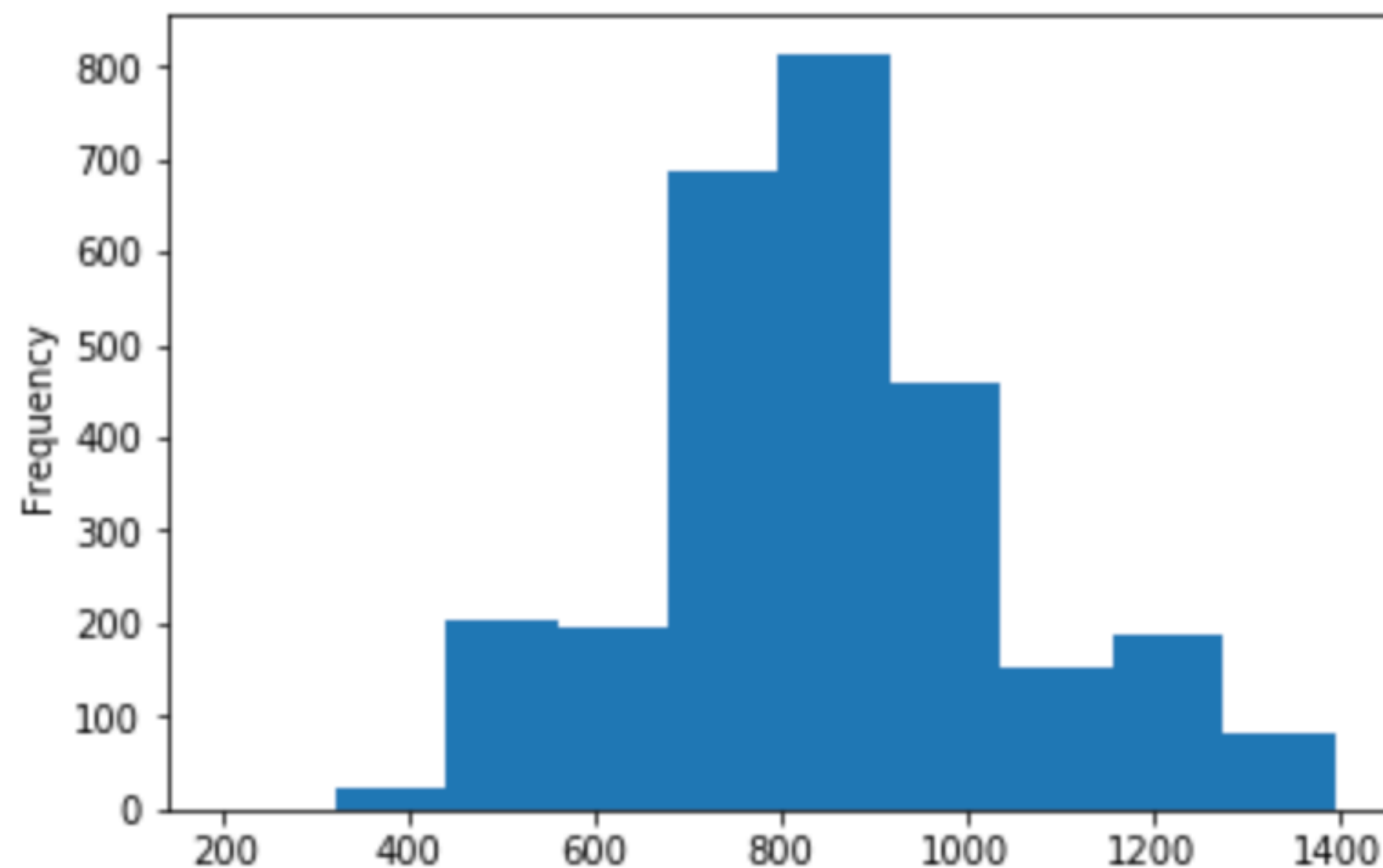
```
df_orders.TYPE.value_counts(normalize=True).plot(kind='barh')
```



Histograms

Shows the distribution of values the variables can take. Useful for continuous data.

```
df_summarized_orders.NUM_ORDERS.plot(kind='hist')
```



```
df_summarized_orders.describe()
```

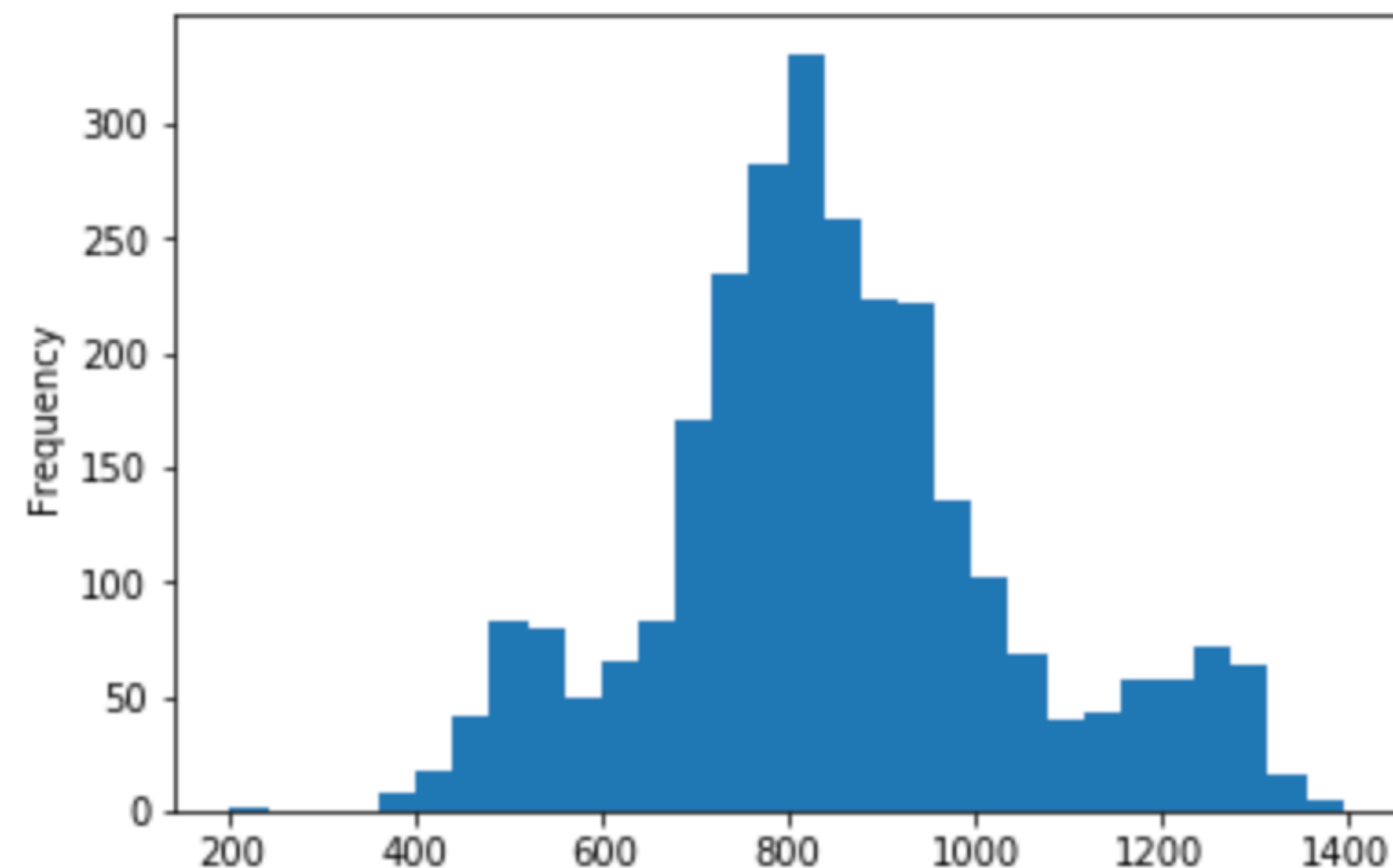
✓ 0.0s

	NUM_ORDERS	PERC_DELIVERY
count	2806.000000	2806.000000
mean	850.756949	0.116700
std	195.490367	0.055078
min	200.000000	0.000000
25%	739.000000	0.090095
50%	833.000000	0.104963
75%	949.000000	0.152853
max	1396.000000	0.360215

Adjust histogram bins

Bins represent the range of values for each bar in a histogram.

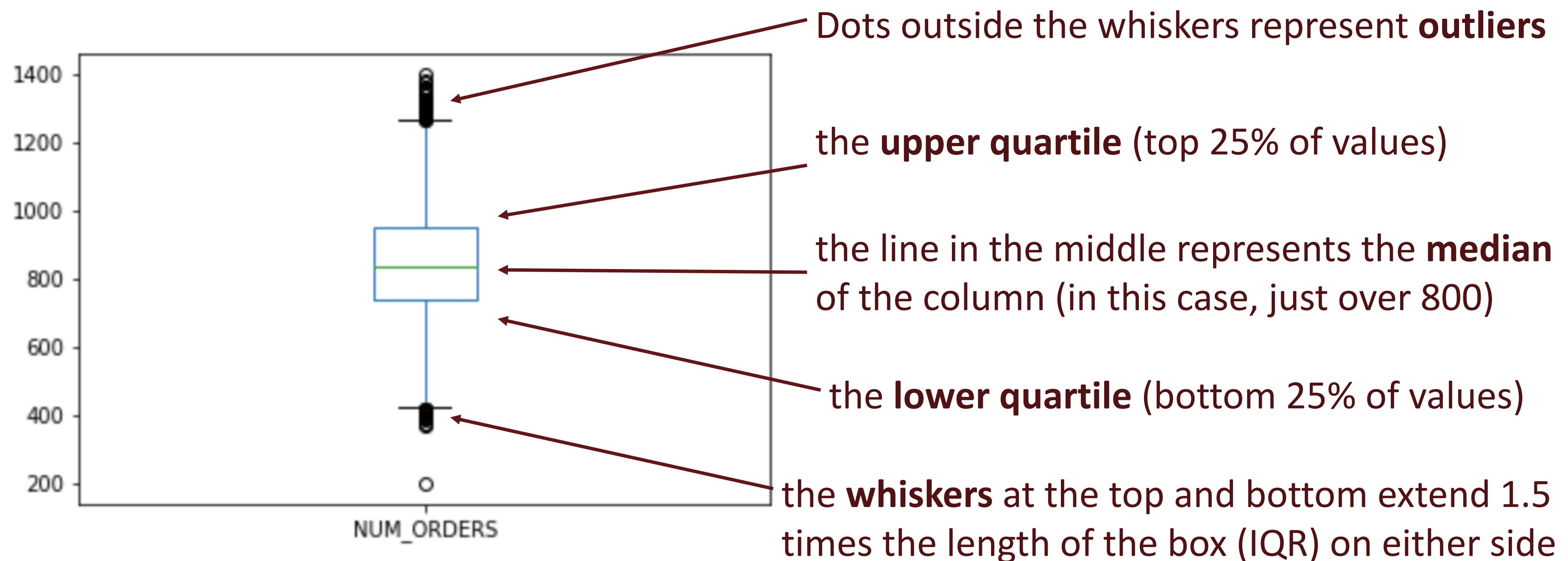
```
df_summarized_orders.NUM_ORDERS.plot(kind='hist', bins=30)
```



Boxplots

Also shows the spread of data, but in a more summarized view.

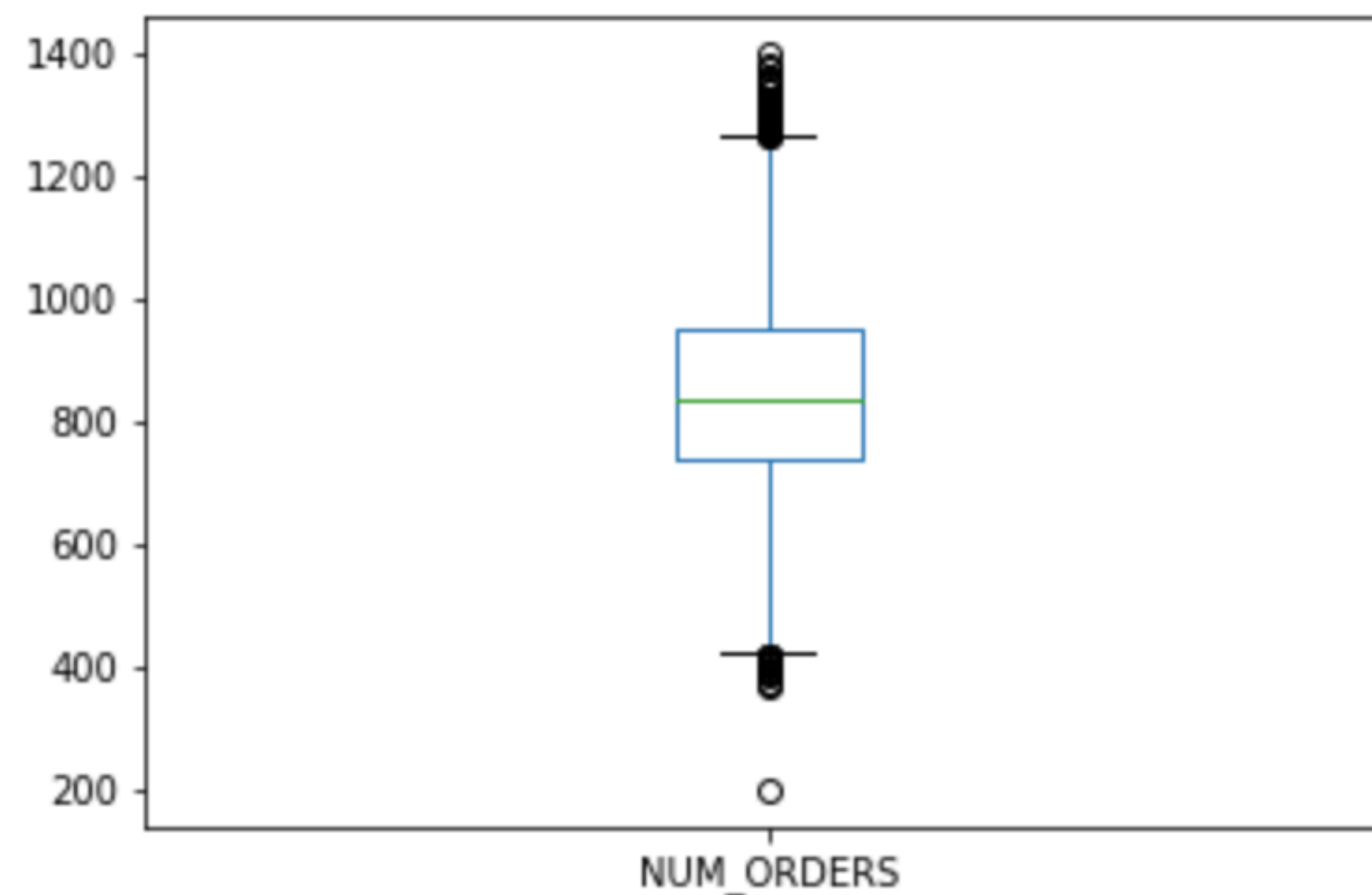
```
df_summarized_orders.NUM_ORDERS.plot(kind='box')
```



Boxplots

Also shows the spread of data, but in a more summarized view.

```
df_summarized_orders.NUM_ORDERS.plot(kind='box')
```



```
df_summarized_orders.describe()
```

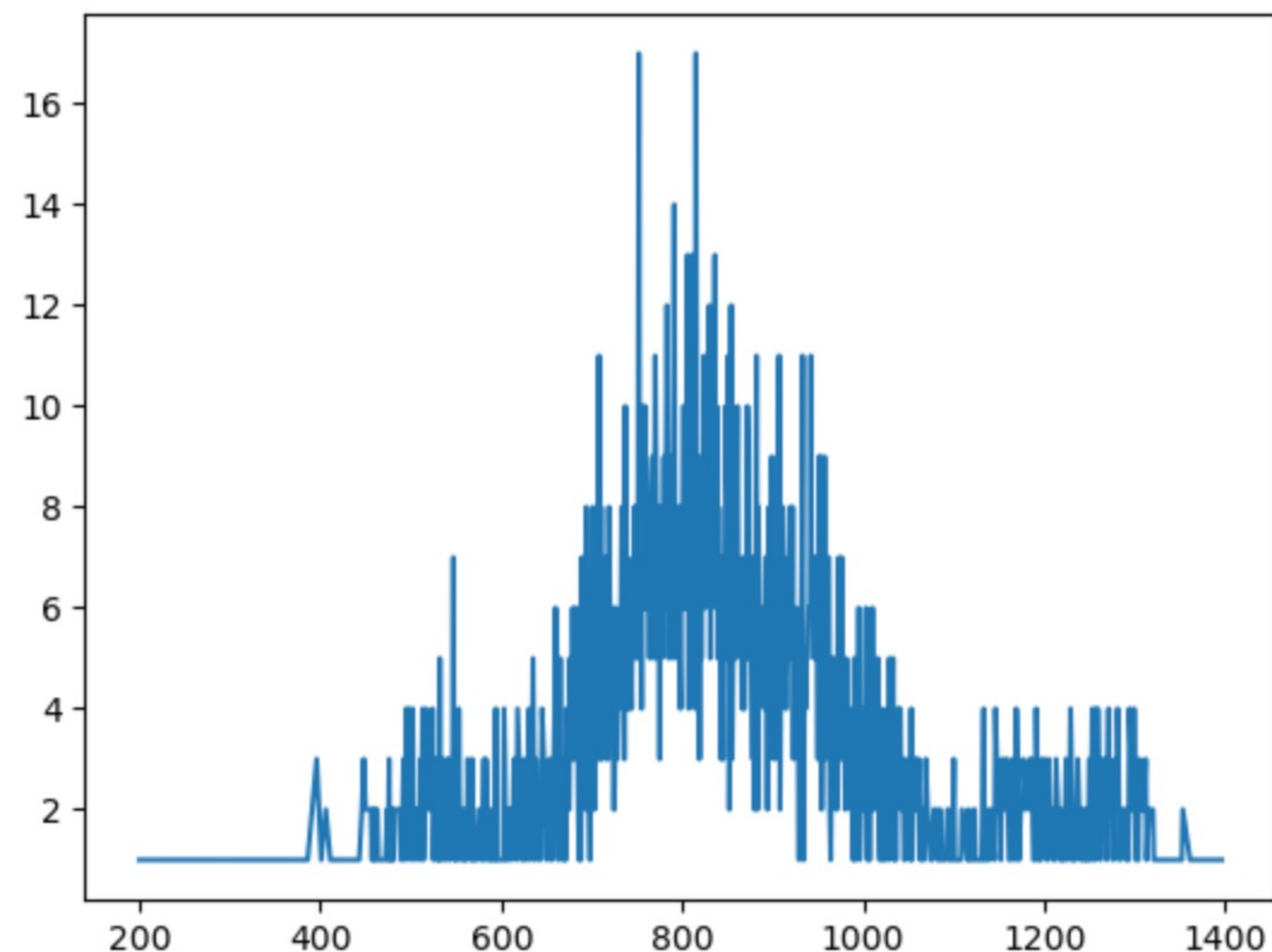
✓ 0.0s

	NUM_ORDERS	PERC_DELIVERY
count	2806.000000	2806.000000
mean	850.756949	0.116700
std	195.490367	0.055078
min	200.000000	0.000000
25%	739.000000	0.090095
50%	833.000000	0.104963
75%	949.000000	0.152853
max	1396.000000	0.360215

Line chart

Also useful for continuous data & to show trends over time.

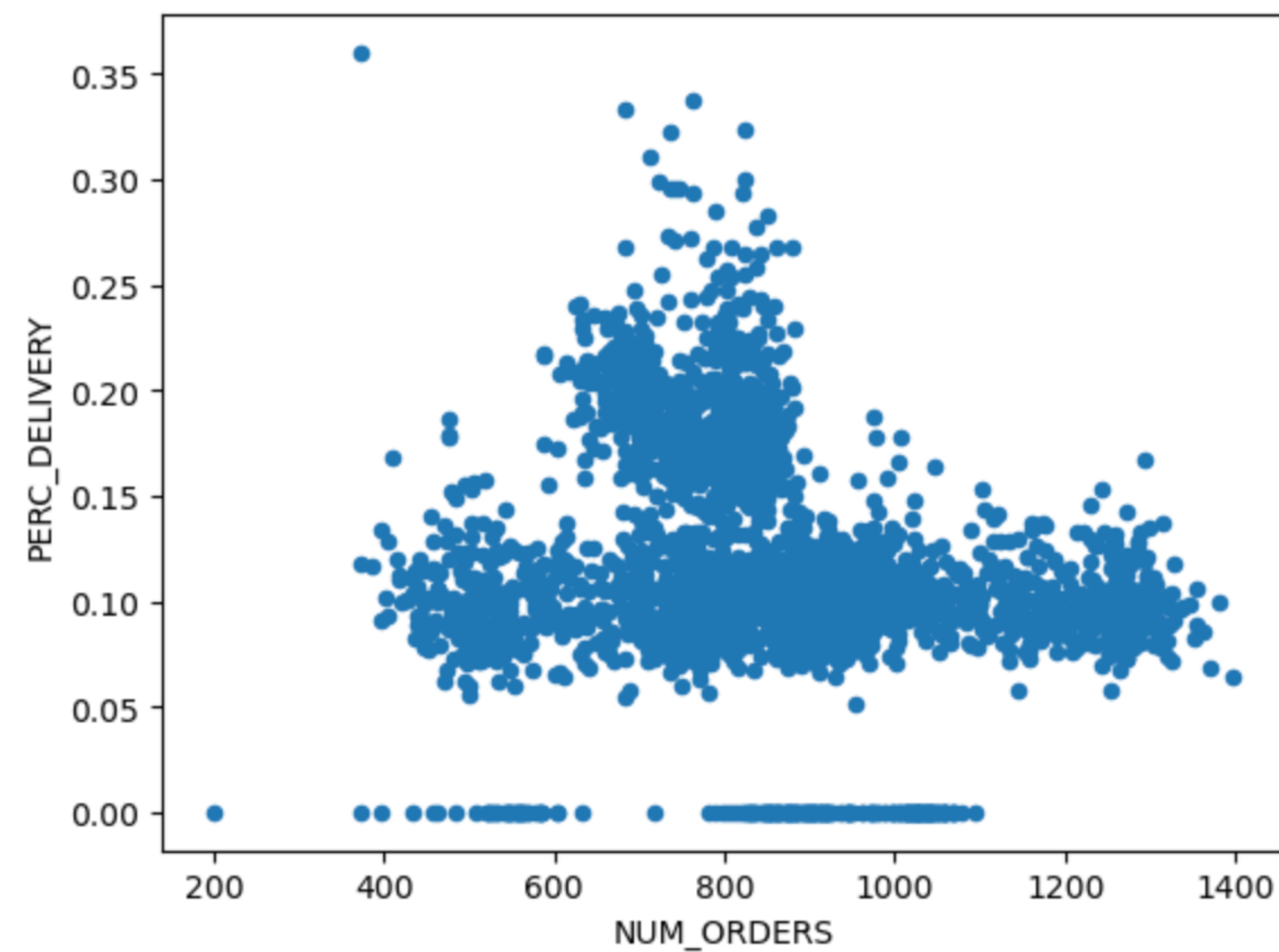
```
df_summarized_orders.NUM_ORDERS.value_counts().sort_index()  
    .plot(kind='line')
```



Scatterplot

Used to evaluate relationships

```
df_summarized_orders.plot(kind='scatter',  
x="NUM_ORDERS", y="PERC_DELIVERY")
```



Pair plots

Also useful to evaluate relationships
between two variables

```
import seaborn as sns
```

seaborn library adds advanced
plotting functionality to Python

```
sns.jointplot(x='NUM_ORDERS',  
              y='PERC_DELIVERY',  
              data=df_summarized_orders,  
              kind='kde')
```

Pair plots

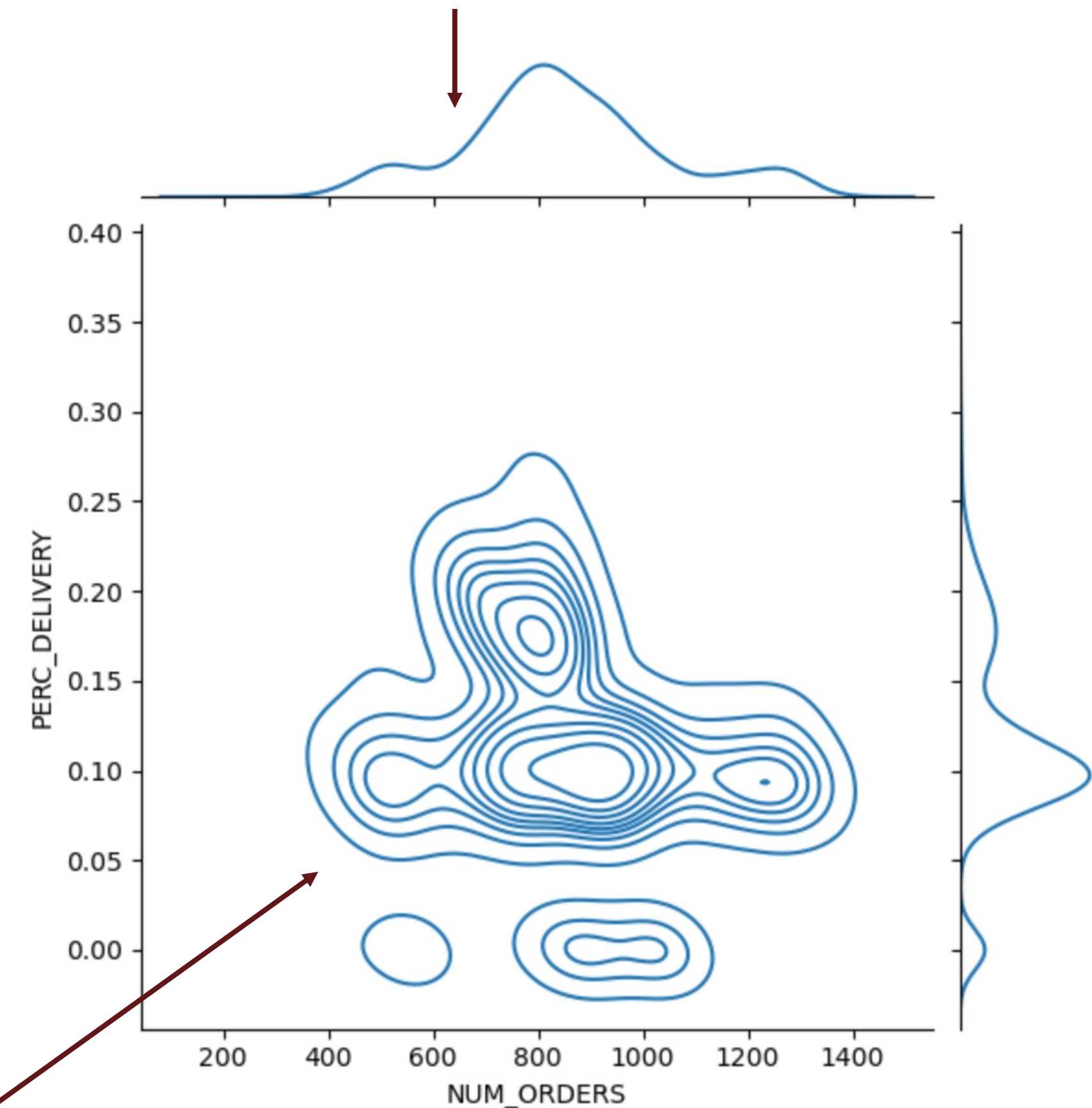
Also useful to evaluate relationships between two variables

```
import seaborn as sns
```

```
sns.jointplot(x='NUM_ORDERS',  
             y='PERC_DELIVERY',  
             data=df_summarized_orders,  
             kind='kde')
```

The shape and closeness of the lines at any point indicates how many points occur in a given area

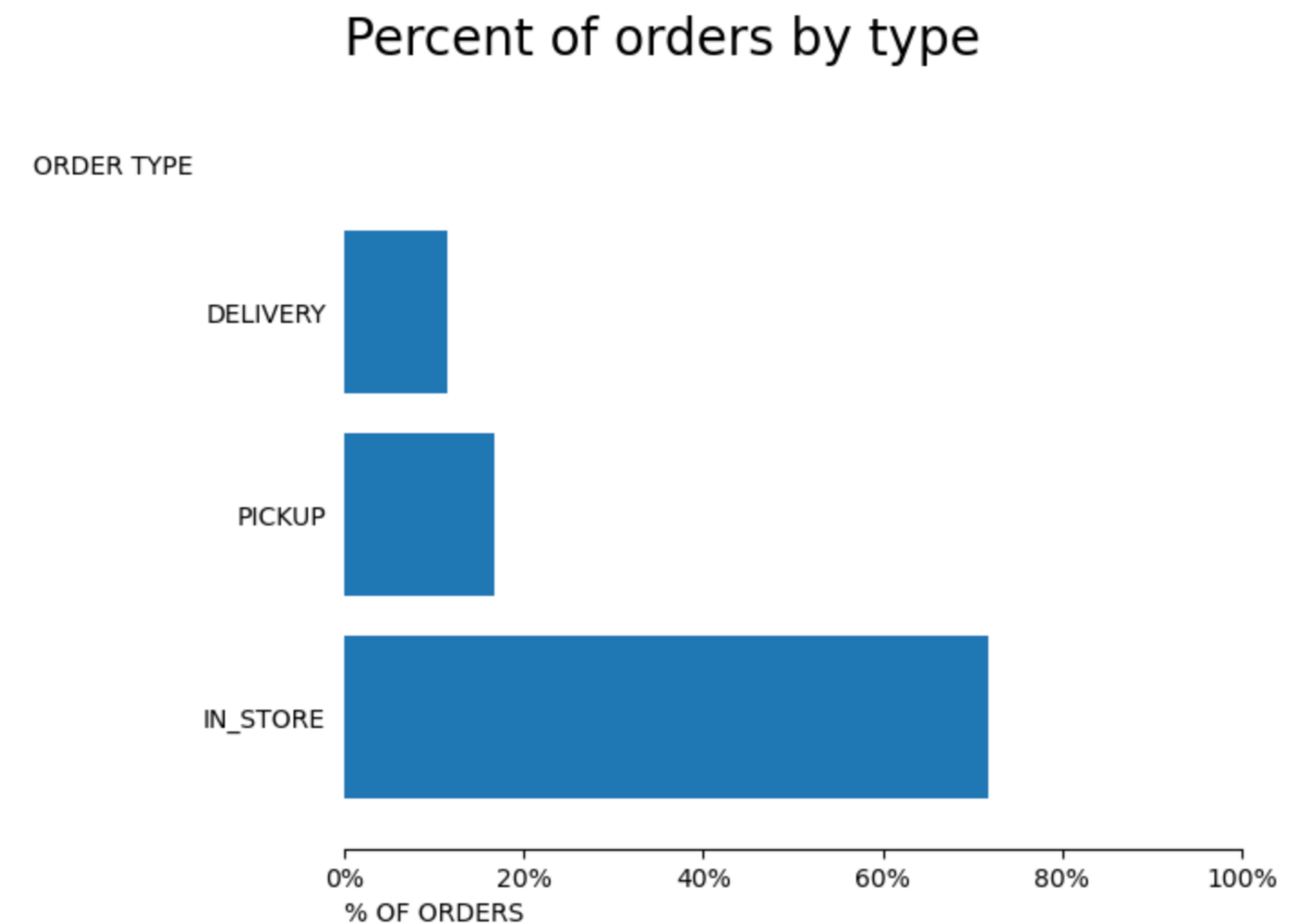
The plots in the margins are *density plots* (a continuous version of the histograms)



Create well designed graphs

Important when communicating findings
(not as necessary when exploring data)

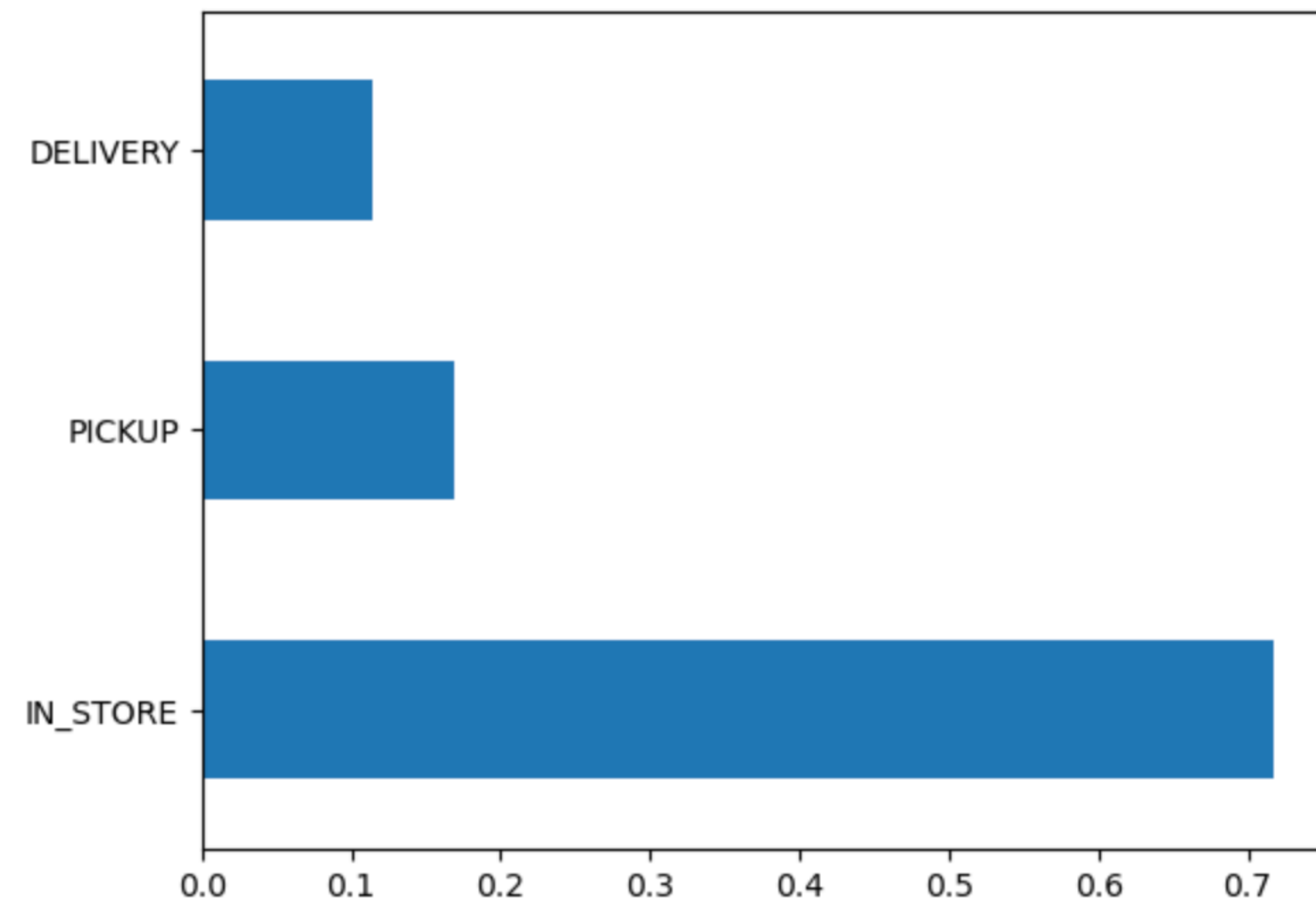
- Remove borders, gridlines & heavy shading
- Include a clear chart title
- Add clear, legible axis labels
- Consider the amount of data & level of detail



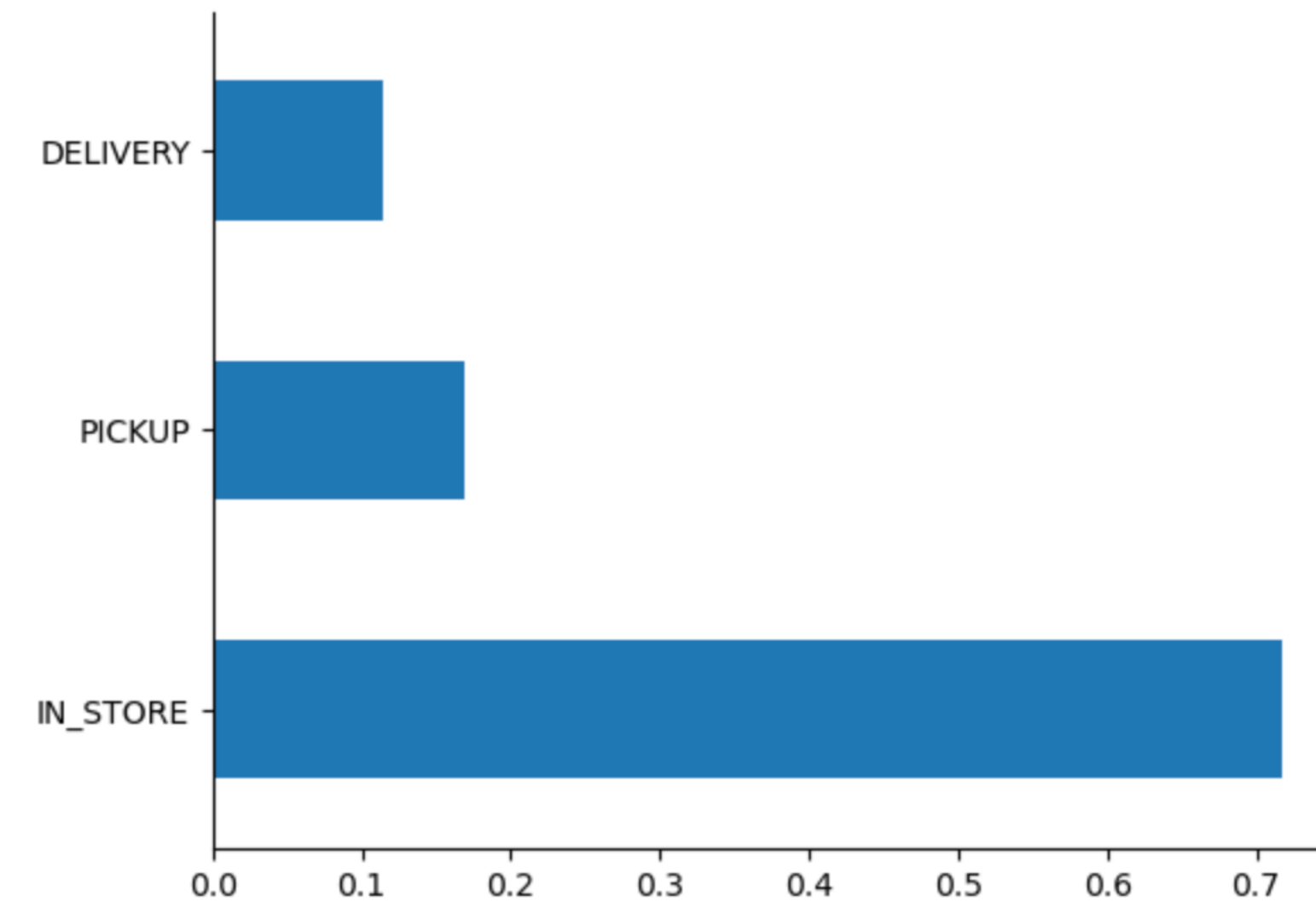
Remove the border

```
df_orders.TYPE.value_counts(normalize=True).plot(kind='barh')  
sns.despine() #remove border
```

DEFAULT

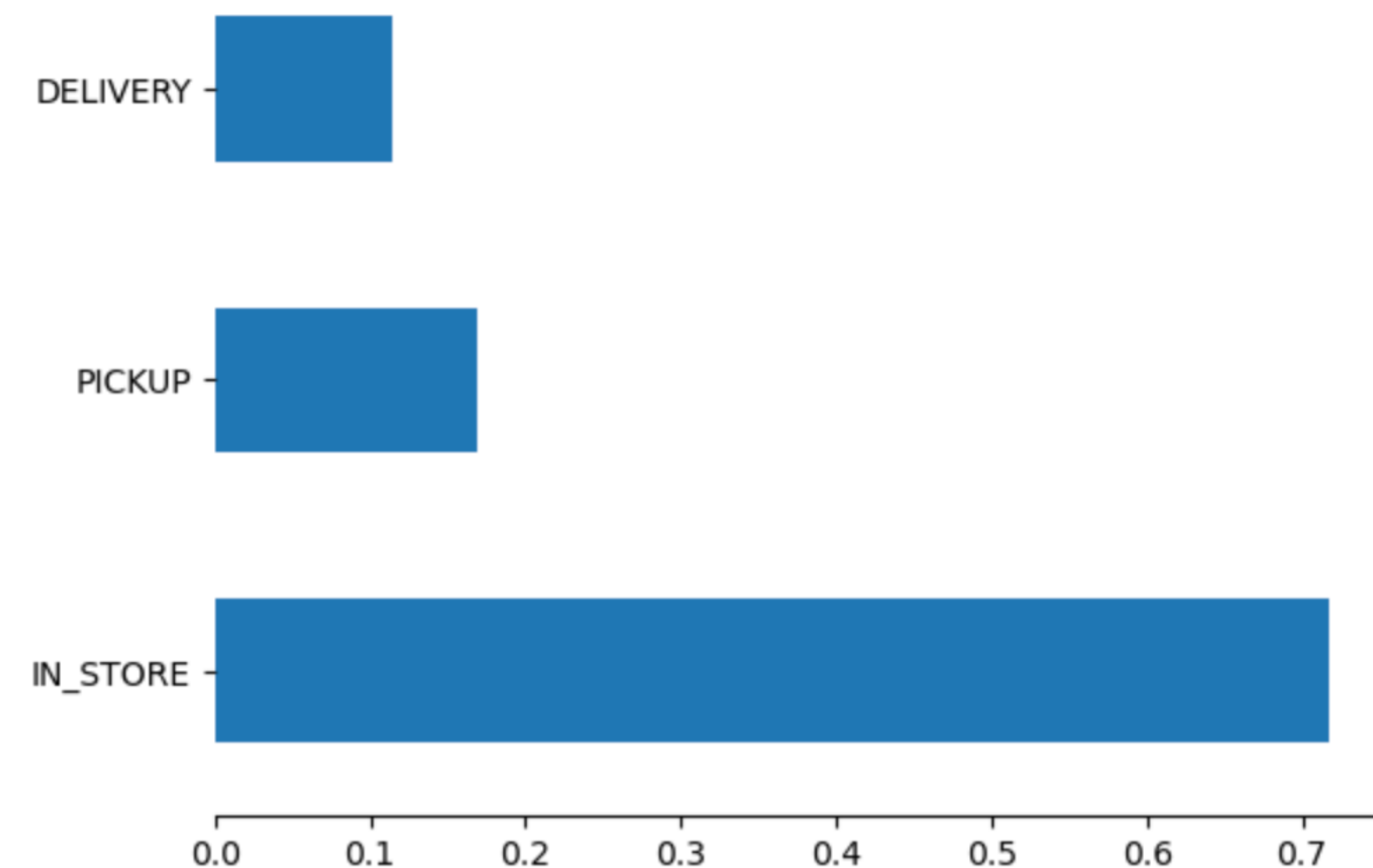


WITHOUT BORDER



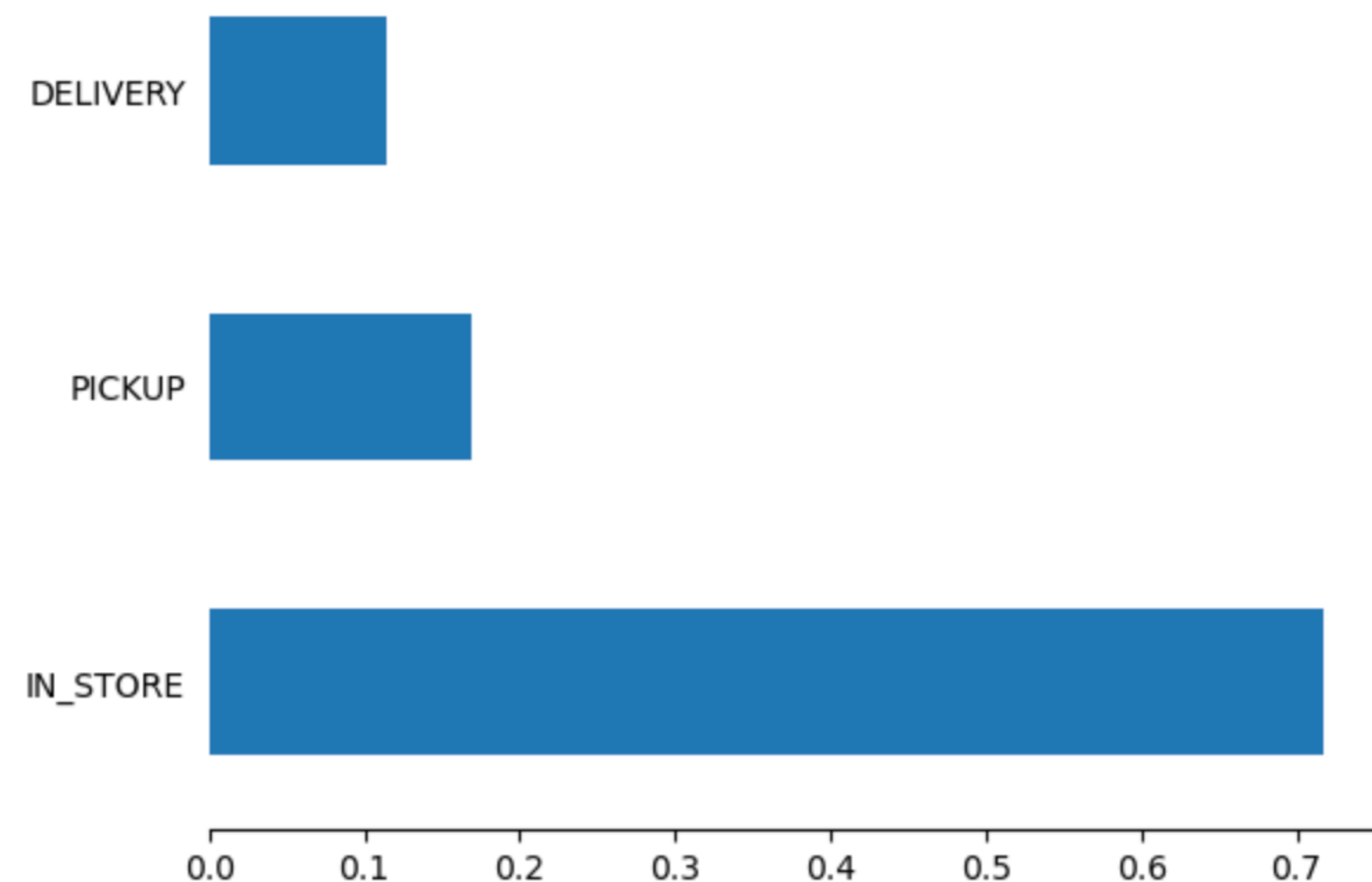
Remove specific borders

```
df_orders.TYPE.value_counts(normalize=True).plot(kind='barh')  
sns.despine() #remove border  
plt.gca().spines["left"].set_visible(False) #can also remove a specific border
```



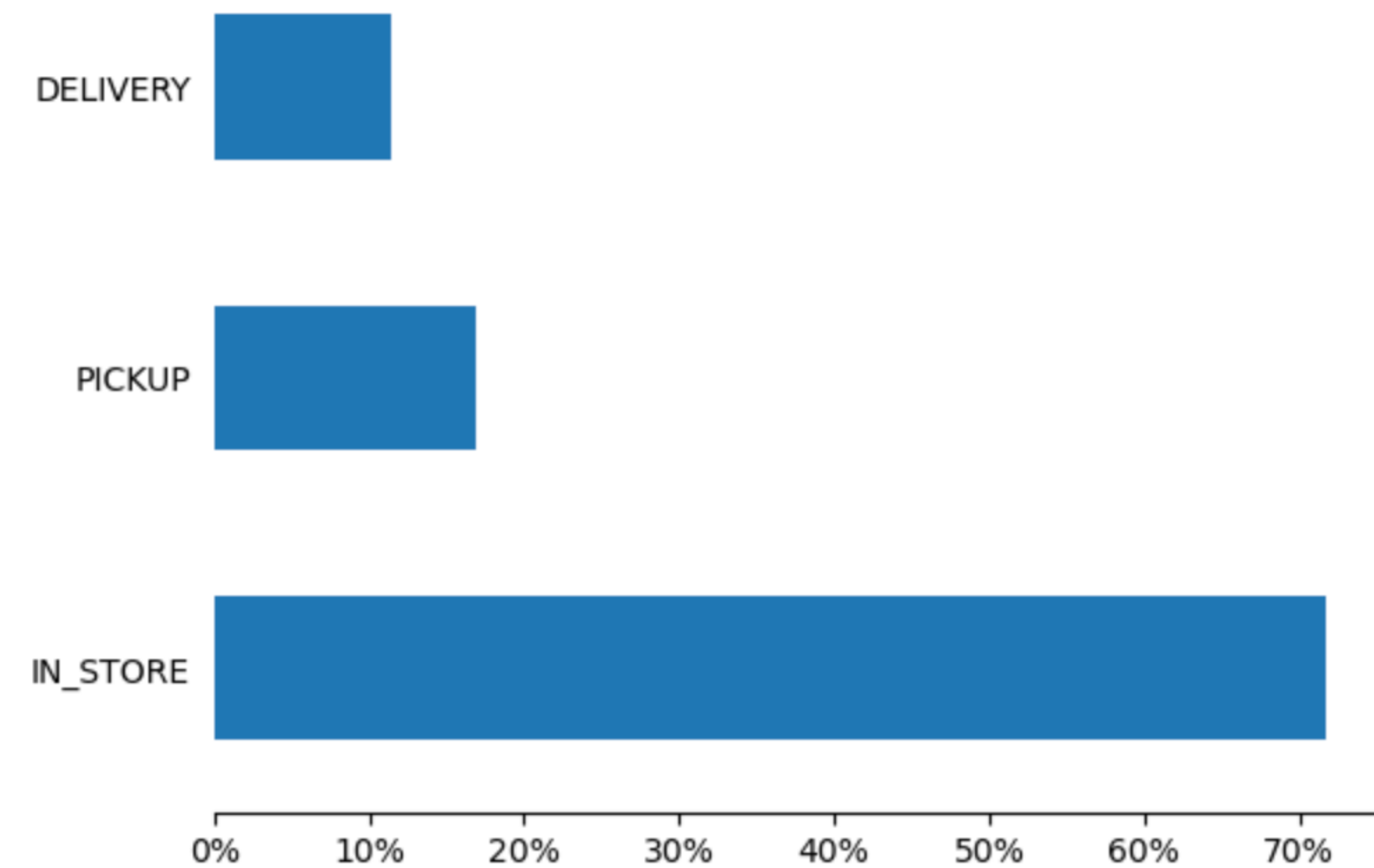
Remove tick marks

```
df_orders.TYPE.value_counts(normalize=True).plot(kind='barh')  
sns.despine() #remove border  
plt.gca().spines["left"].set_visible(False) #can also remove a specific border  
plt.tick_params(left = False) #remove y-axis ticks; the x-axis here would be bottom  
= False but in this case they are a good reference
```



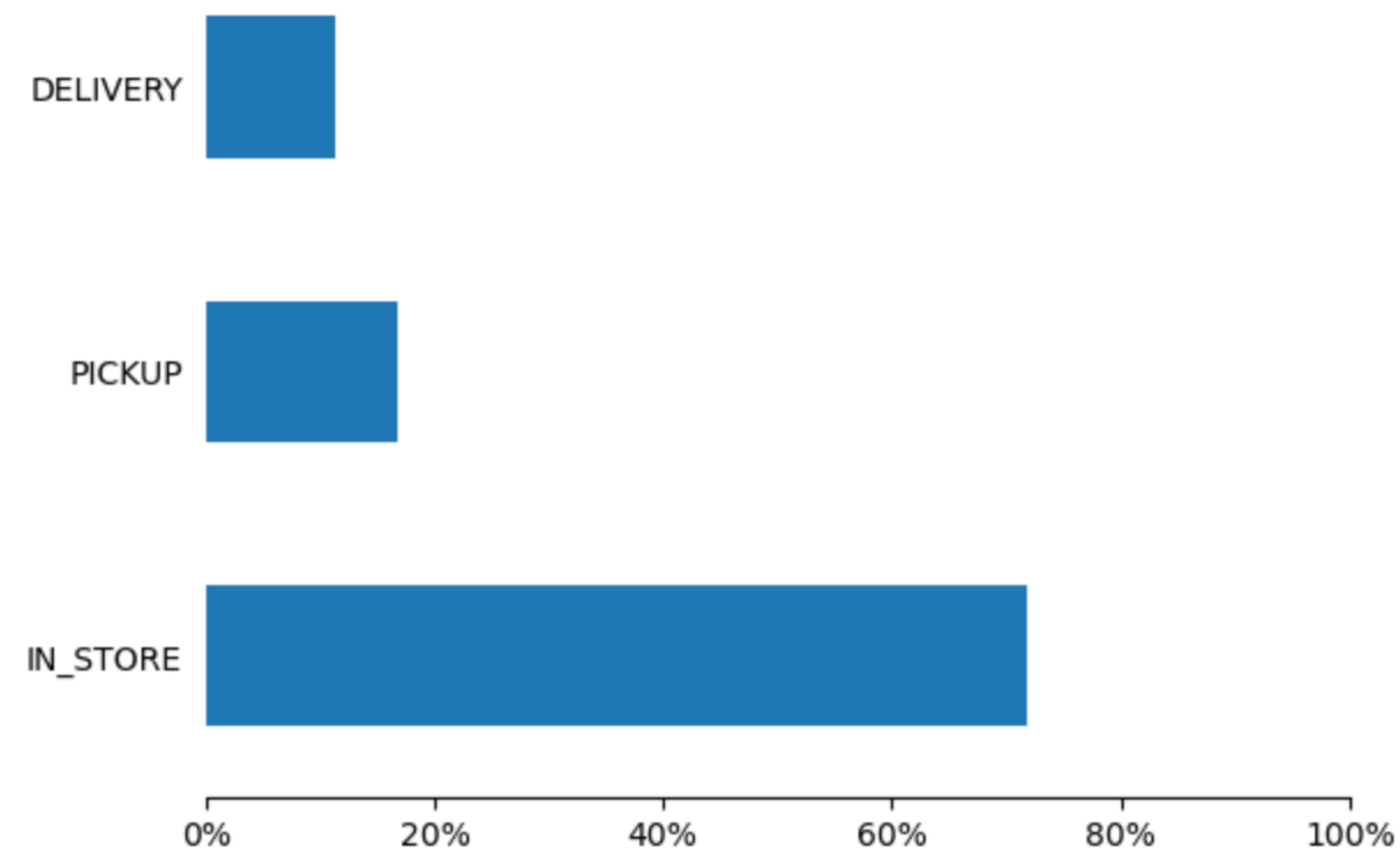
Format x-axis labels

```
from matplotlib.ticker import PercentFormatter  
plt.gca().xaxis.set_major_formatter(PercentFormatter(1)) #format as percent
```



Adjust the x-axis range

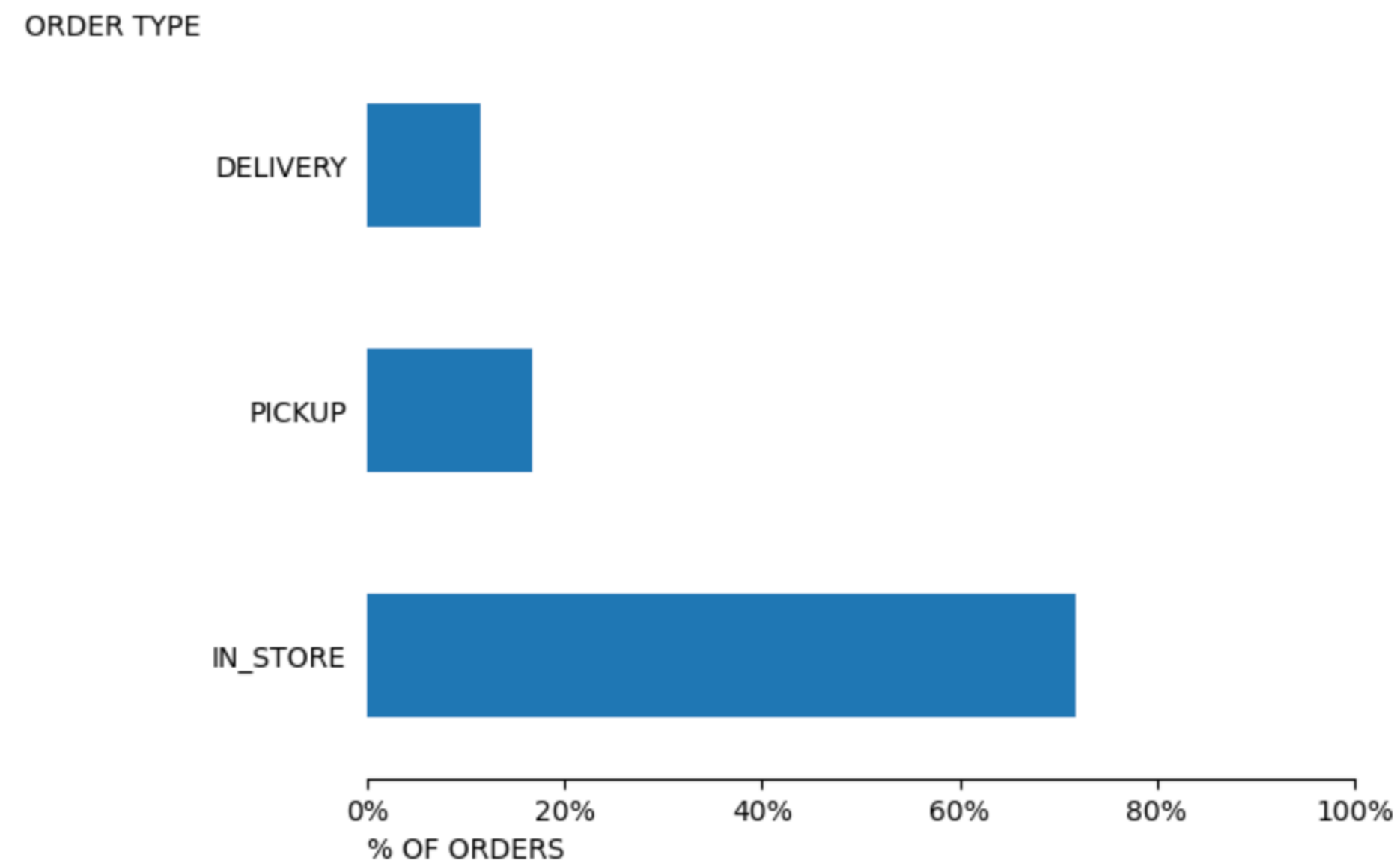
```
from matplotlib.ticker import PercentFormatter
plt.gca().xaxis.set_major_formatter(PercentFormatter(1)) #format as percent
plt.xlim(0,1) #set axis range
```



Add axis titles

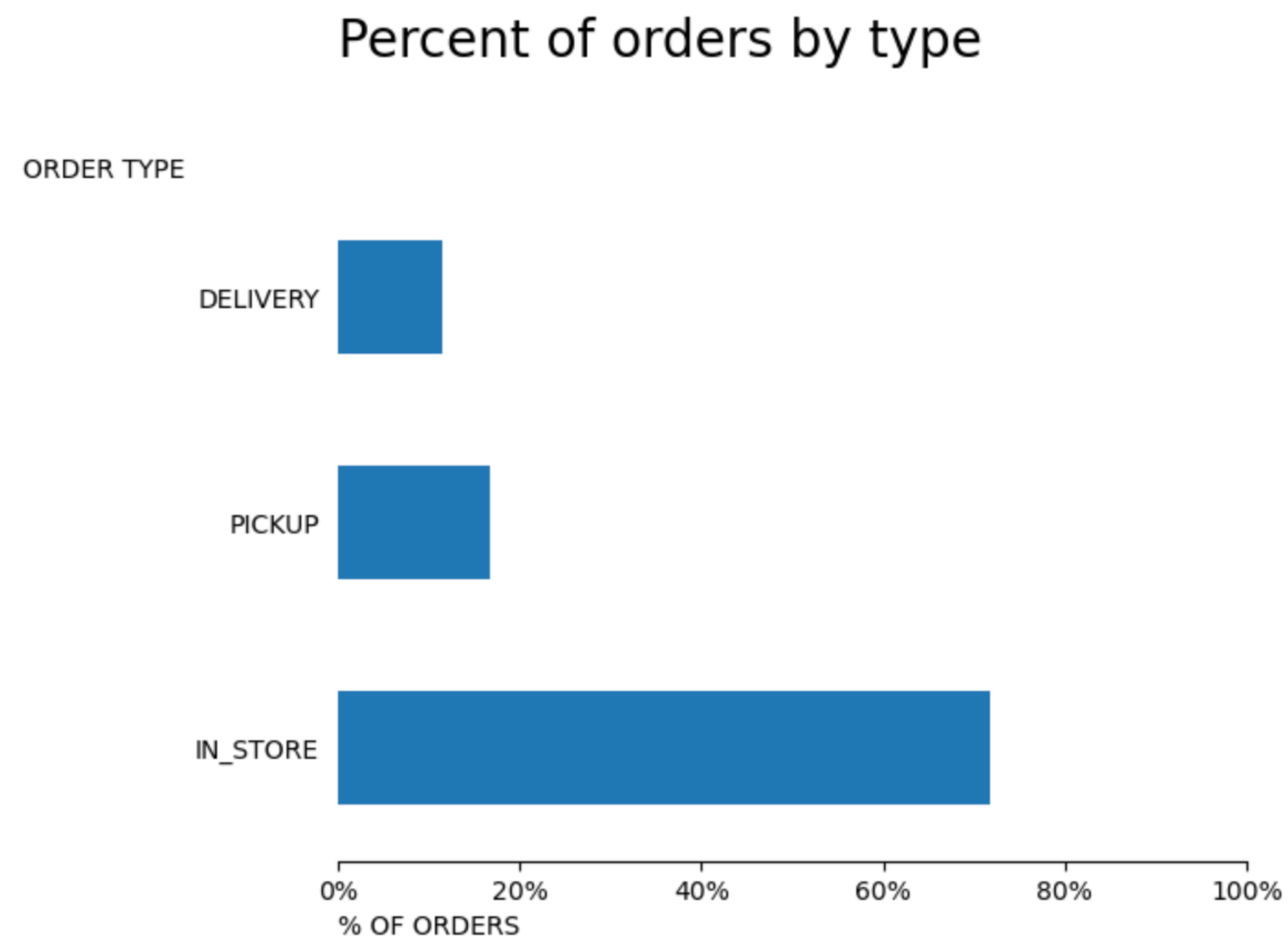
```
plt.xlabel("% OF ORDERS", loc = "left") #add x-axis label
```

```
plt.ylabel("ORDER TYPE", loc="top", rotation=0) #add y-axis label
```



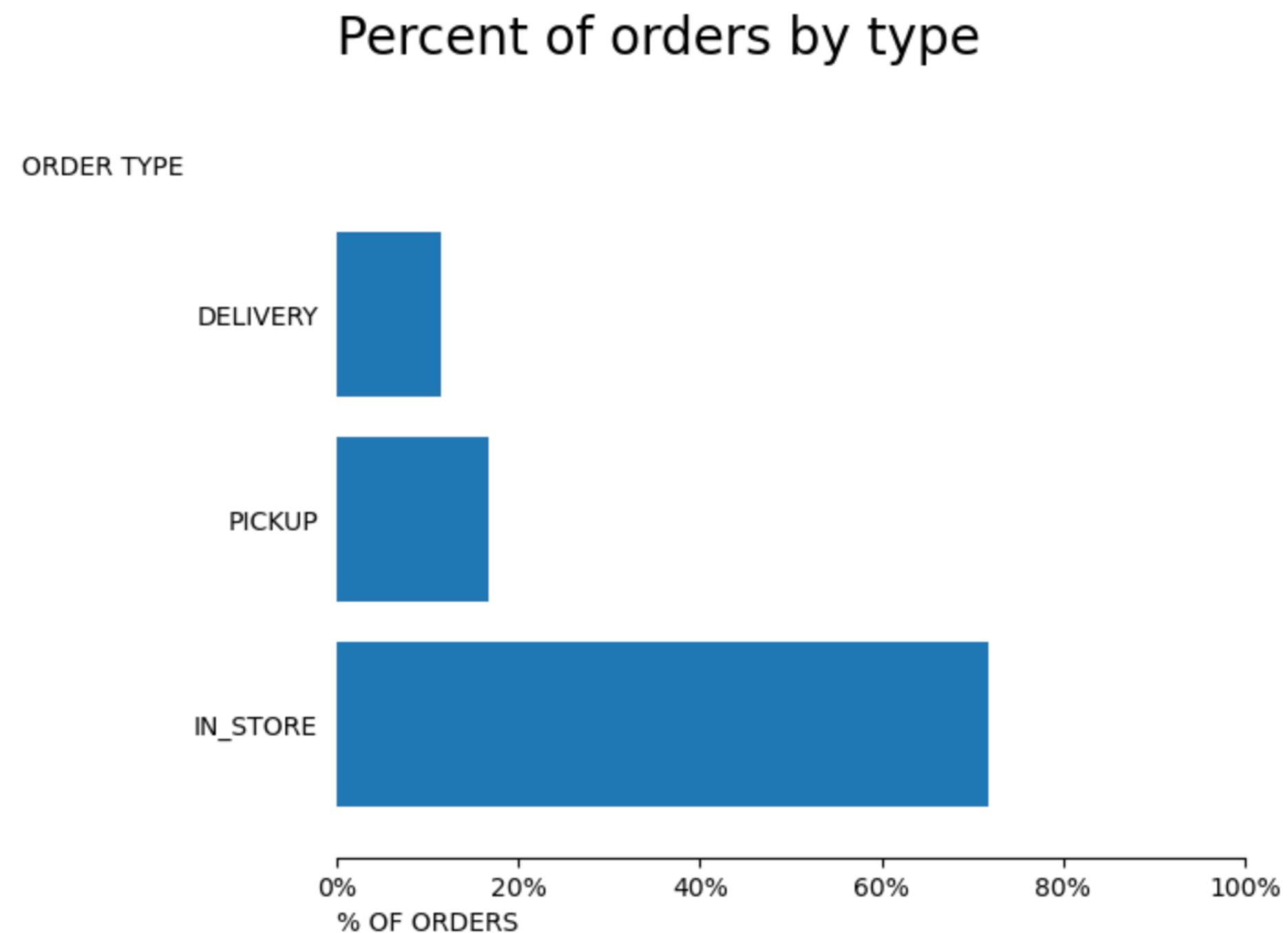
Include a chart title

```
plt.title("Percent of orders by type", loc="left", pad=50, fontsize=20) #add &
format chart title
```



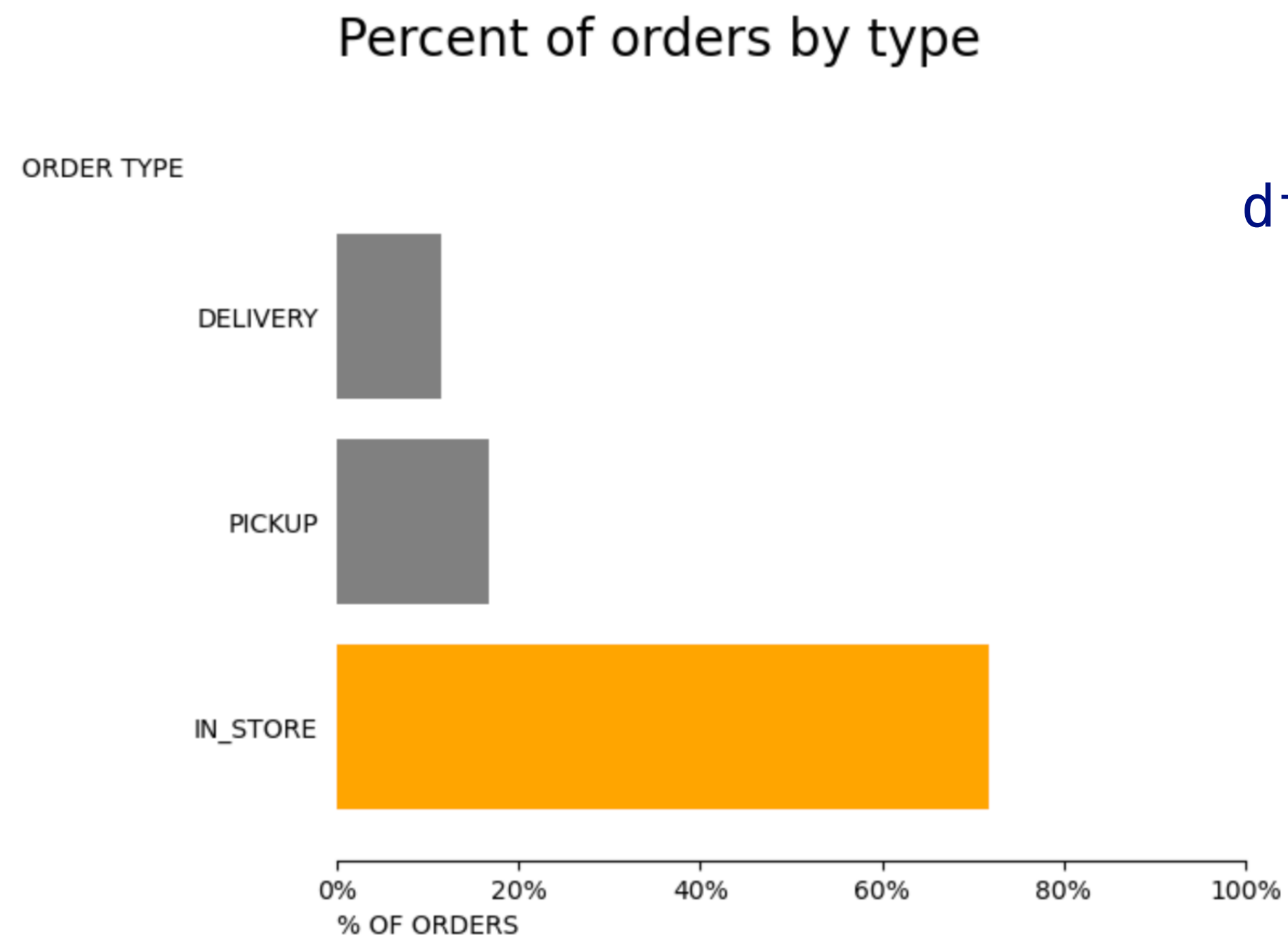
Finally, adjust the bar width

```
df_orders.TYPE.value_counts(normalize=True).plot(kind='barh', width=.8)
```



Focus attention

Intentionally draw your audience's eyes to certain information



```
df_orders.TYPE.value_counts(normalize=True)
    .plot(kind='barh'
, width=.8
, color=['orange', 'grey', 'grey'])
```

Focusing techniques

- Sparing use of color
- Size
- Position
- Bold font for emphasis
- Limited data markers & labels

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

We'll resume in
5 minutes...

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

We'll resume in
4 minutes...

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

We'll resume in
3 minutes...

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

We'll resume in
2 minutes...

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

We'll resume in
1 minute...

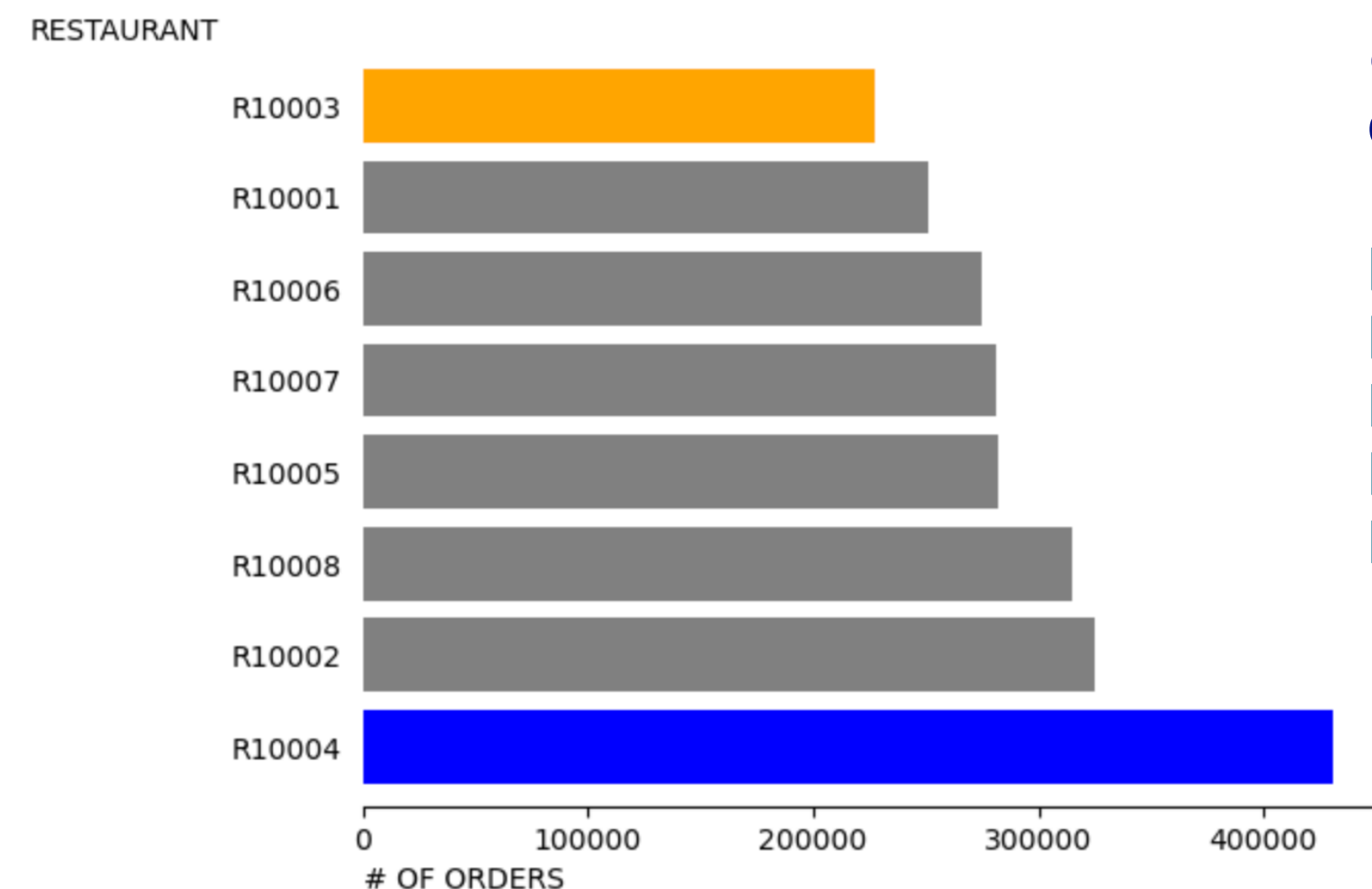
Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.

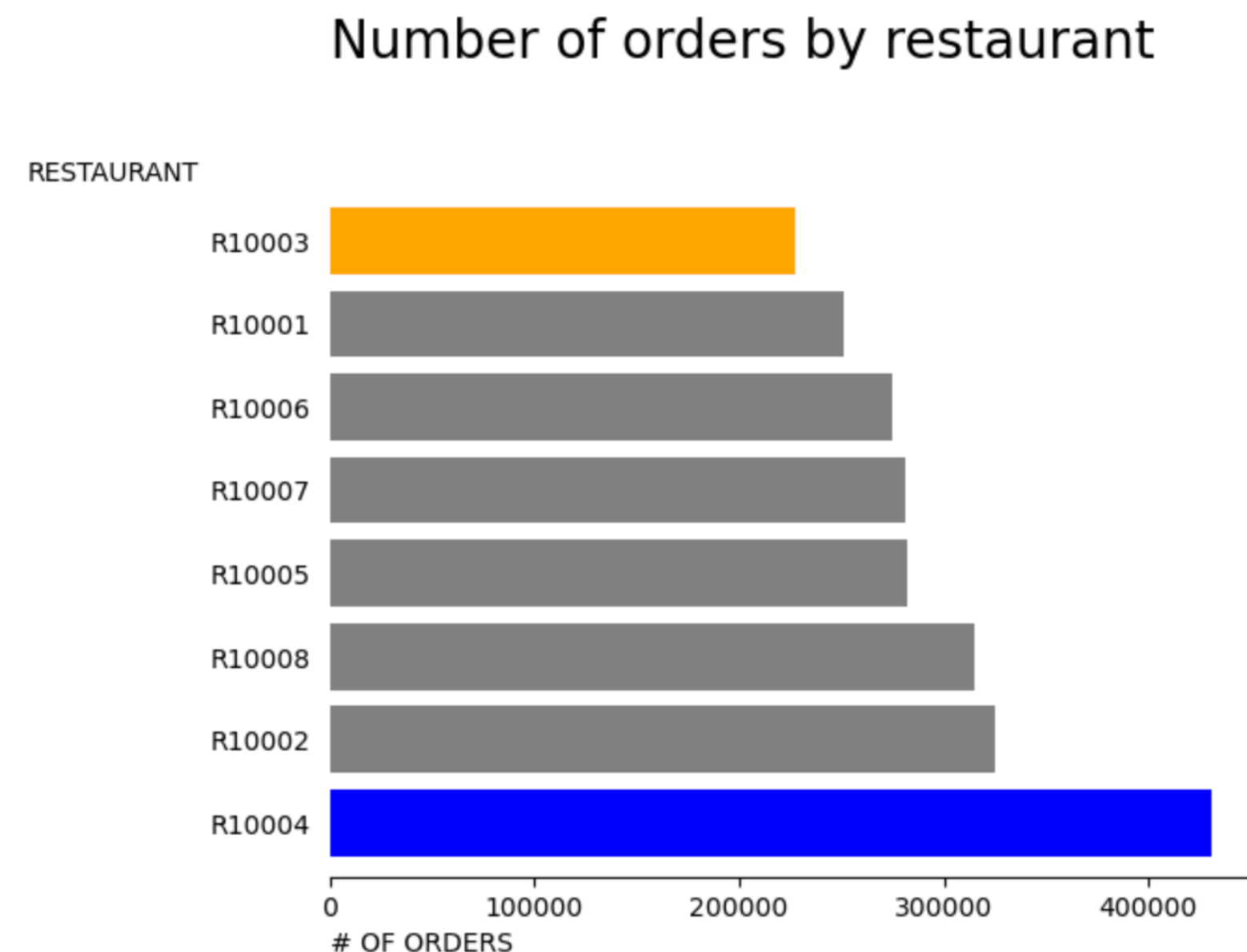
Number of orders by restaurant



```
colors = ['blue', 'grey', 'grey', 'grey', 'grey', 'grey', 'grey', 'orange']
df_orders.RESTAURANT_ID.value_counts().plot(kind='barh', width=.8, color=colors)
sns.despine()
plt.gca().spines["left"].set_visible(False)
plt.tick_params(left = False)
plt.xlabel("# OF ORDERS", loc = "left")
plt.ylabel("RESTAURANT", loc="top", rotation=0)
plt.title("Number of orders by restaurant", loc="left", pad=50, fontsize=20)
```

Challenge 3: top/bottom restaurants

Using the *df_orders* table, create a visual to see which restaurant has the highest number of orders and which has the least.



```
df_restaurants.head(8)
```

✓ 0.0s

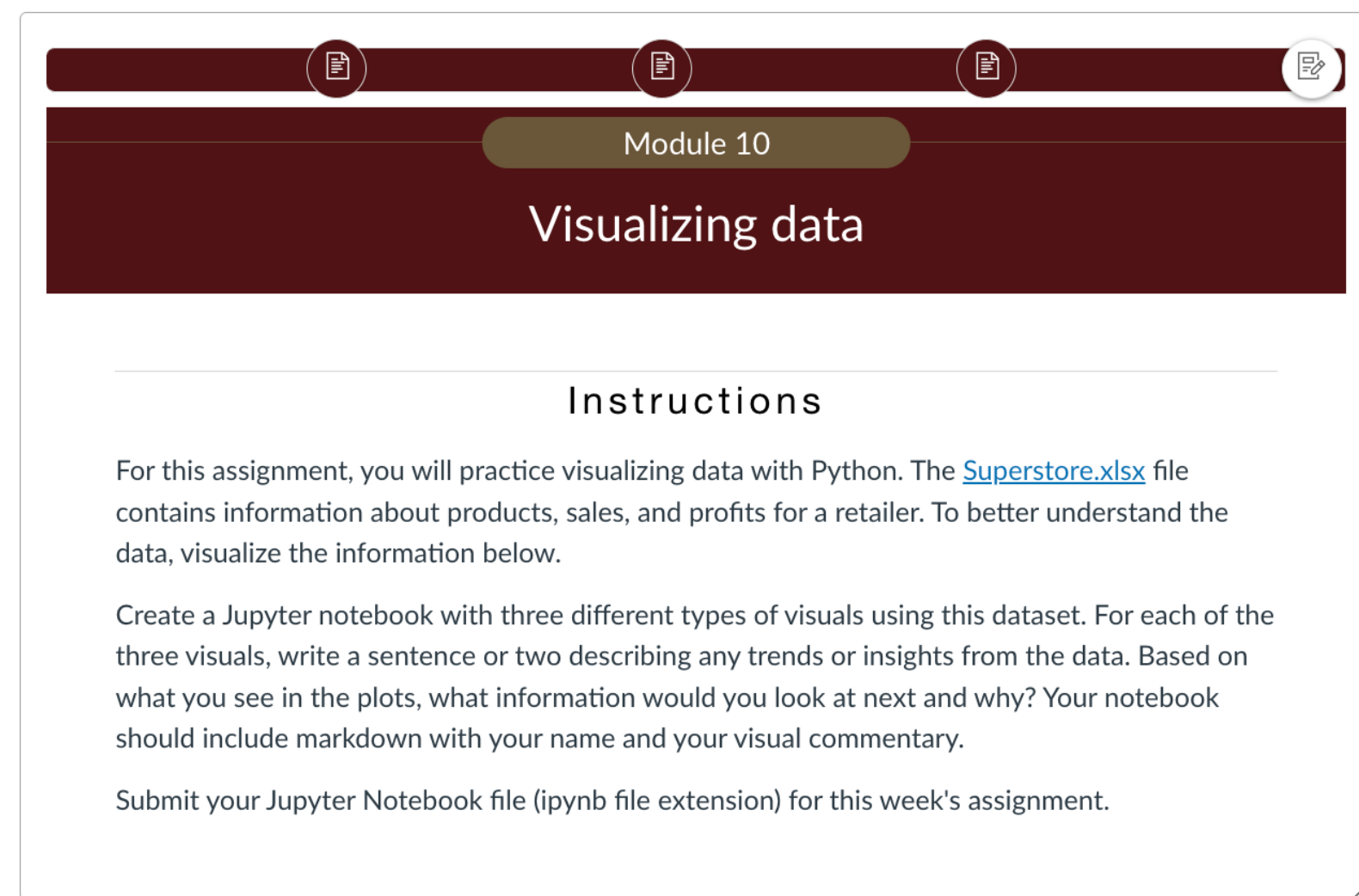
	RESTAURANT_ID	NAME	ADDRESS
0	R10001	Columbia	2884 Broadway, New York, NY 10025
1	R10002	Midtown	1379 6th Ave, New York, NY 10019
2	R10005	Flatiron	40 W 25th St, New York, NY 10010
3	R10008	Williamsburg	45 S 3rd St, Brooklyn, NY 11249
4	R10004	NYU	109 Macdougall St, New York, NY 10012
5	R10003	Bryant Park	70 W 40th St, New York, NY 10018
6	R10006	Upper East Side	1045 Lexington Ave, New York, NY 10021
7	R10007	Upper West Side	2140 Broadway, New York, NY 10023

Quick recap

- We can explore data by calculating descriptive statistics
 - describe, sum, min, max, mean, median
- We can look at specific values in a dataframe
 - head, unique, isin, numpy.where
- We can filter with logic statements
- We can summarize with value_counts()
- We can visualize with plotting libraries

Next week—assignment

Coding assignment due by next class



The screenshot shows a Jupyter Notebook interface with a dark red header bar. The header bar contains four circular icons: a document, a folder, a search icon, and a share icon. Below the header bar, the text 'Module 10' is displayed in a light green box, followed by 'Visualizing data' in a larger white font. The main content area is titled 'Instructions' and contains three paragraphs of text. The first paragraph states that the assignment involves visualizing data with Python using the 'Superstore.xlsx' file. The second paragraph instructs the user to create a Jupyter notebook with three different types of visuals and to include markdown with their name and visual commentary. The third paragraph states that the Jupyter Notebook file (ipynb) should be submitted for the assignment.

Module 10

Visualizing data

Instructions

For this assignment, you will practice visualizing data with Python. The [Superstore.xlsx](#) file contains information about products, sales, and profits for a retailer. To better understand the data, visualize the information below.

Create a Jupyter notebook with three different types of visuals using this dataset. For each of the three visuals, write a sentence or two describing any trends or insights from the data. Based on what you see in the plots, what information would you look at next and why? Your notebook should include markdown with your name and your visual commentary.

Submit your Jupyter Notebook file (ipynb file extension) for this week's assignment.

REMINDER: homework is an independent effort, not group work

Next week—lecture

Editing DataFrames in Pandas and approaches to address inaccurate and missing data

